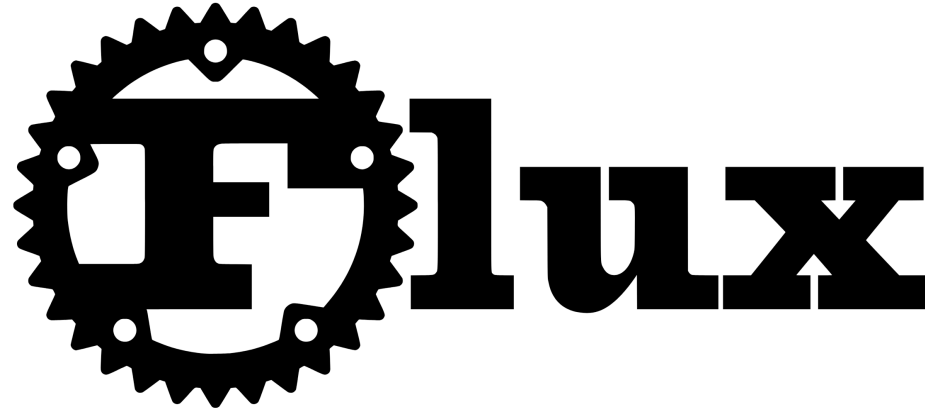


Refinement Types for Verified Systems



Ranjit Jhala, UC San Diego

What *is* Programming Languages Research?



*“We shall make
thoughtcrime
literally impossible:
there will be no words
to express it.”*

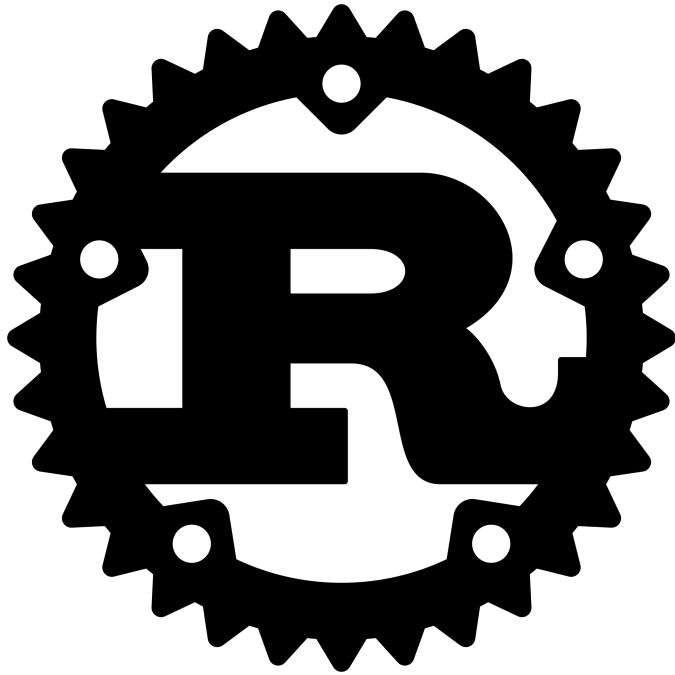
— George Orwell (1984)



*“We shall make
~~thoughtcrime~~ **bugs**
literally impossible:
there will be no words
to express it.”*

— George Orwell (1984)

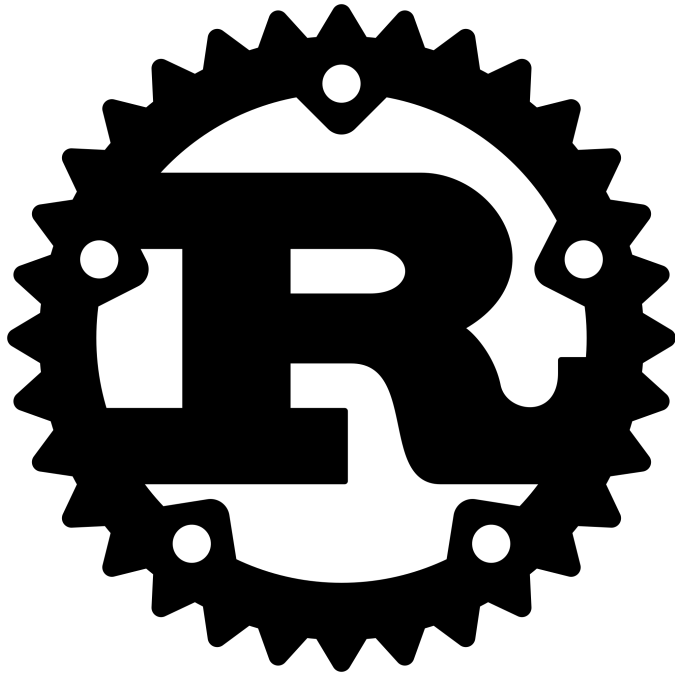
What is Programming Languages Research?



*“We shall make
~~thoughtcrime~~ **bugs**
literally impossible:
there will be no words
to express it.”*

— George Orwell (1984)

What *is* Programming Languages Research?

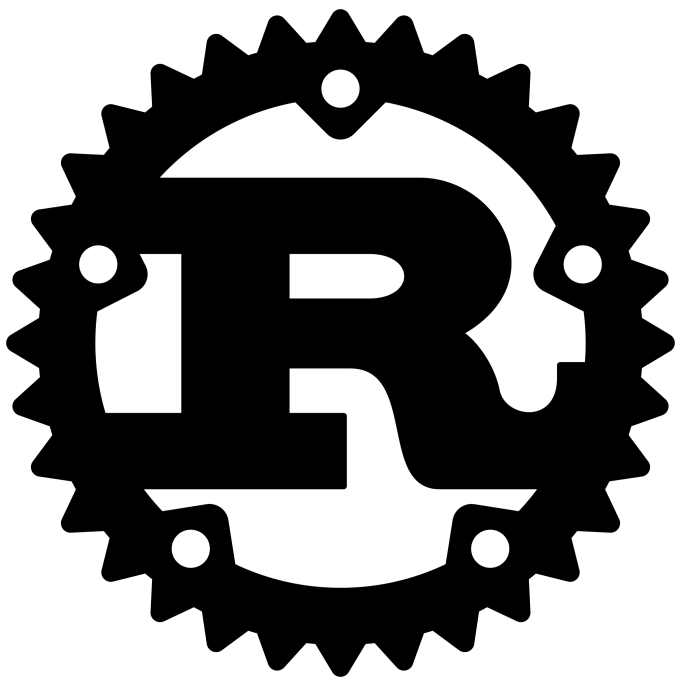


Rust Types Ensure

Memory safety

Race freedom

What is Programming Languages Research?



Google Security Blog

The latest news and insights from Google on security and safety on the Internet

Rust in Android: move fast and fix things

November 13, 2025

We adopted Rust for its security and are seeing a **1000x reduction in memory safety vulnerability density compared to Android's C and C++ code**. But the biggest surprise was Rust's impact on software delivery. With Rust changes having a **4x lower rollback rate** and spending **25% less time in code review**, the safer path is now also the faster one.

What is Programming Languages Research?

Other bugs?

Google Security Blog

The latest news and insights from Google on security and safety on the Internet

Rust in Android: move fast and fix things

November 13, 2025

We adopted Rust for its security and are seeing a **1000x reduction in memory safety vulnerability density compared to Android's C and C++ code**. But the biggest surprise was Rust's impact on software delivery. With Rust changes having a **4x lower rollback rate** and spending **25% less time in code review**, the safer path is now also the faster one.

What *is* Programming Languages Research?

Array Overflows

Other bugs?

What *is* Programming Languages Research?

Array Overflows

Integer Overflows

Other bugs?

What *is* Programming Languages Research?

Array Overflows

Integer Overflows

User def. invariants

Other bugs?

What is Programming Languages Research?

Array Overflows

Integer Overflows

User def. invariants

Security Requirements

Other bugs?

What is Programming Languages Research?

Array Overflows

Integer Overflows

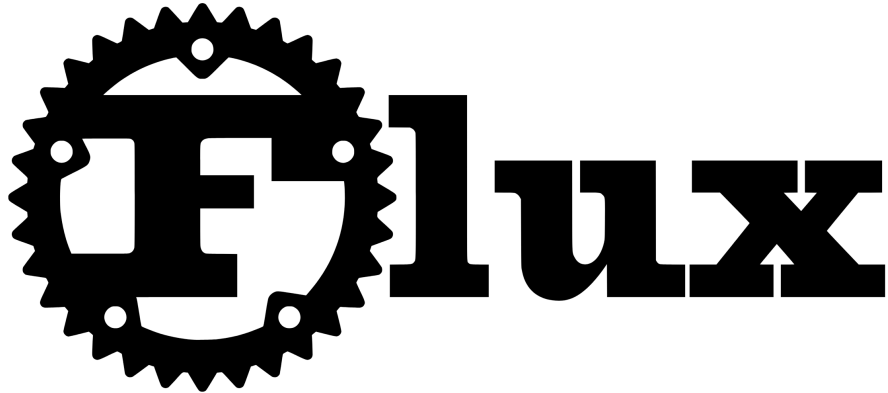
User def. invariants

Security Requirements

Functional Correctness

Other bugs?

Refinement Types for Verified Systems



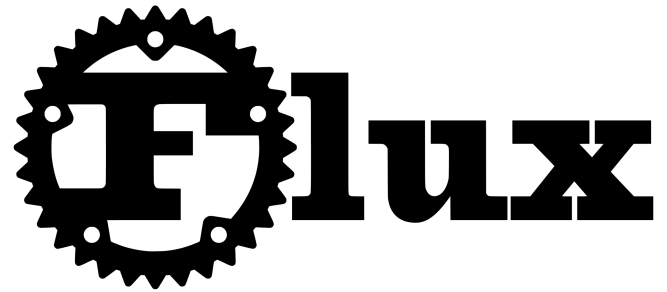
Array Overflows

Integer Overflows

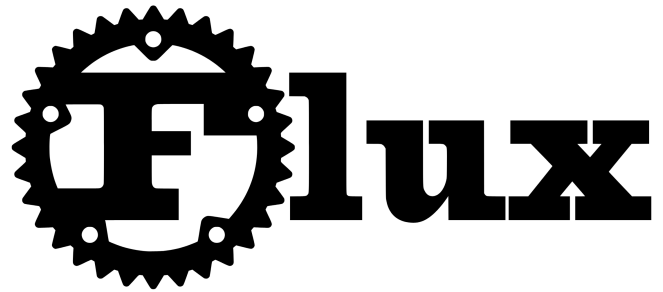
User def. invariants

Security Requirements

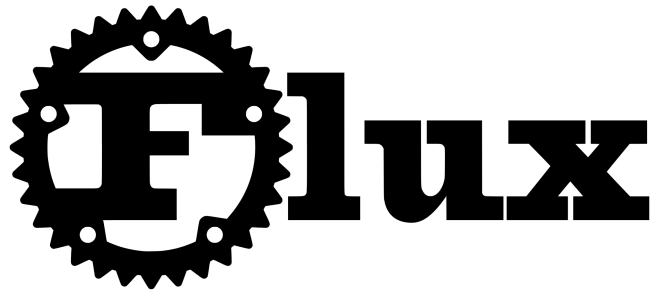
Functional Correctness



1. Refinement



1. Refinement 2. Types



1. Refinement 2. Types 3. Verified Systems

1. Refinement

1. Refinement

Index specifies *single value*

Index specifies *single value*

B [*v*]

Base Type *Refine* Index

Index specifies *single value*

i32[5]

The *singleton* **i32** that is equal to 5

Index specifies *single value*

`bool[true]`

The *singleton* `bool` that is equal to `true`

Index specifies *single value*

```
fn tt() → bool[true] {  
    1 < 2  
}
```

Output type specifies *Postcondition*

A function that *always returns* **true**

Index specifies *single value*

```
fn ff() → bool[false] {  
    2 < 1  
}
```

Output type specifies *Postcondition*

A function that *always returns* false

Index specifies *single value*

```
fn twelve() → i32[12] {  
    4 + 8  
}
```

Output type specifies *Postcondition*

A function that *always returns* 12

Index specifies *single value*

```
fn assert(b:bool[true]){{}}
```

Input type specifies *Precondition*

A function that *requires* input be **true**

Index specifies *single value*

```
fn assert(b:bool[true]){}  
  
...  
assert(1 < 2);  
assert(10 < 2); // flux error!
```

Input type specifies *Precondition*

A function that *requires* input be **true**

Index specifies *single value*

Constants are *boring*!

Index specifies *single value*

Constants are *boring!*

Parameters abstract over *input values*

Parameters abstract over *input values*

Refinement parameters

```
forall<n: int> fn (i32[n]) → bool[n > 0]
```

Parameters abstract over *input values*

Refinement parameters

```
forall<n: int> fn (i32[n]) → bool[n > 0]
```

Declare with @-syntax

```
fn (i32[@n]) → bool[n > 0]
```

Parameters abstract over *input values*

Refinement parameters

```
forall<n: int> fn (i32[n]) → bool[n > 0]
```

Declare with @-syntax

```
fn (i32[@n]) → bool[n > 0]
```

Or desugar from Rust

```
fn (n:i32) → bool[n > 0]
```

Parameters abstract over *input values*

Output's type *depends on input*

```
fn is_pos(n:i32) → bool[n>0] {  
    n > 0  
}
```

Parameters abstract over *input values*

Output's type *depends on input*

```
fn is_pos(n:i32) → bool[n>0] {  
    n > 0  
}  
  
...  
assert(is_pos(5));           // ok
```

Parameters abstract over *input values*

Output's type *depends on input*

```
fn is_pos(n:i32) → bool[n>0] {  
    n > 0  
}  
  
...  
assert(is_pos(5));           // ok  
assert(is_pos(5 - 8));      // error
```

Parameters abstract over *input values*

Output's type *depends on input*

```
fn incr(n:i32) → i32[n+1] {  
  n + 1  
}
```

Parameters abstract over *input values*

Output's type *depends on input*

```
fn incr(n:i32) → i32[n+1] {  
    n + 1  
}  
  
...  
assert(incr(5 - 5) > 0); // ok
```

Parameters abstract over *input values*

Output's type *depends on input*

```
fn incr(n:i32) → i32[n+1] {  
    n + 1  
}  
  
...  
assert(incr(5 - 5) > 0); // ok  
assert(incr(5 - 6) > 0); // error
```

1. Refinement

Index specifies *single value*

Parameters abstract over *inputs*

Index specifies *single value*

B[*v*]

Index specifies *single value*

B [*v*]

But what if we *don't know* the exact value?

1. Refinement

Index specifies *single value*

Parameters abstract over *inputs*

Existentials represent *sets of values*

Existentials represent *sets of values*

Existentials represent *sets of values*

$$B \{ v : p(v) \}$$

Base Type Constraint

Existentials represent *sets of values*

$$\mathbf{B} \{ v : p(v) \}$$

Set of $\mathbf{B}$ values whose index v satisfies $p(v)$

Existentials represent *sets of values*

i32 { $v : 0 \leq v$ }

i32 values that are *non-negative*

Existentials represent *sets of values*

`use {v : v < n}`

`use` values *less than* `n`

Existentials represent *sets of values*

`abs` returns a non-negative `i32`

```
fn abs(n:i32) → i32{v:0 ≤ v} {  
  if n < 0 {  
    0 - n  
  } else {  
    n  
  }  
}
```

Existentials represent *sets of values*

abs returns a non-negative **i32**

```
fn abs(n:i32) → i32{v:0 ≤ v} {  
  if n < 0 {  
    0 - n  
  } else {  
    n  
  }  
}  
...  
assert(abs(n) ≥ 0);
```

Exercise: How can we *fix* the spec for *abs*?

Existentials represent *sets of values*

abs returns a non-negative **i32**

```
fn abs(n:i32) → i32{v:0 ≤ v} {  
  if n < 0 {  
    0 - n  
  } else {  
    n  
  }  
}  
...  
assert(abs(n) ≥ 0);  
assert(abs(n) >= n); // EX: How to fix?
```

Exercise: How can we *fix* the spec for *abs*?

Existentials represent *sets of values*

get requires a valid index!

```
fn get<T>(x: &[T], i: usize) → &T
{
    &x[i] // EX: How to fix?
}
```

Exercise: What is a suitable type for the input *i*?

Existentials represent *sets of values*

get requires a valid index!

```
fn get<T>(x: &[T][@n], i: usize{v:v<n}) → &T
{
    &x[i]
}
```

Precondition: *i less than size of slice n*

1. Refinement

Index specifies *single value*

Parameters abstract over *inputs*

Existentials represent *sets of values*

1. Refinement

Index specifies *single value*

Parameters abstract over *inputs*

Existentials represent *sets of values*

Ownership enables sound *type updates*

Ownership enables sound *type updates*

Ownership enables sound *type updates*

At *owned locations* and *mutable references*

Type updates at *owned locations*

Type updates at *owned locations*

```
let mut x = 0;    // x : i32[0]  
assert(x == 0);
```

Exclusive ownership allows strong updates

Type updates at *owned locations*

```
let mut x = 0;      // x : i32[0]
assert(x == 0);
x += 10;            // x : i32[10]
assert(x == 10);
```

Exclusive ownership allows strong updates

Type updates at *owned locations*

```
let mut x = 0;      // x : i32[0]
assert(x == 0);
x += 10;            // x : i32[10]
assert(x == 10);
x += 10;            // x : i32[20]
assert(x == 20);
```

Exclusive ownership allows strong updates

Type updates at *mutable references*

Type updates at *mutable references*

```
fn incr(x: &mut i32[@n]) ensures x: i32[n+1] {  
    *x += 1;  
}
```

Type updates at *mutable references*

```
fn incr(x: &mut i32[@n]) ensures x: i32[n+1] {  
    *x += 1;  
}
```

```
fn test() {  
    let mut z = 1; // z: i32[1]
```

Type updates at *mutable references*

```
fn incr(x: &mut i32[@n]) ensures x: i32[n+1] {  
    *x += 1;  
}  
  
fn test() {  
    let mut z = 1; // z: i32[1]  
    incr(&z);  
    assert(z == 2); // z: i32[2]  
}
```

Sound as Rust ensures `&mut` has *no aliases*

Ownership enables sound *type updates*

At *owned locations* and *mutable references*

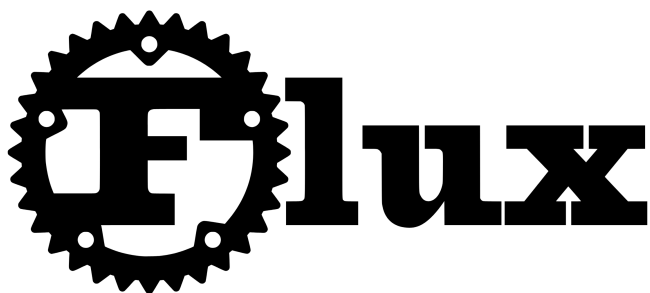
1. Refinement

Index specifies *single value*

Parameters abstract over *inputs*

Existentials represent *sets of values*

Ownership enables sound *type updates*

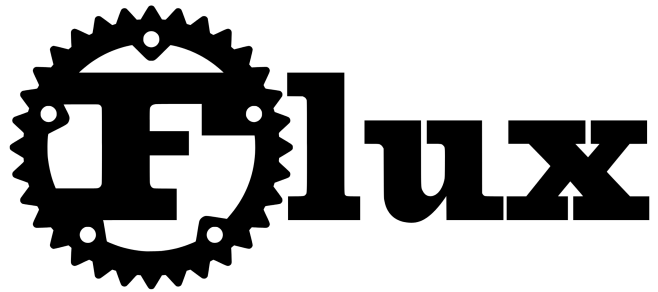


1. Refinement

2. Types

3. Verified Systems

Index, Exist & Update



1. Refinement

2. Types

3. Verified Systems

Index, Exist & Update

2. Types

2. *Types*

Compositional specification & verification

2. *Types*

Compositional specification & verification

“Make illegal states unrepresentable”

2. *Types*

struct

“Product” types

enum

“Sum” types

2. *Types*

struct

“Product” types

enum

“Sum” types

struct

Example: *Refined Vectors*

struct: *Refined Vectors*

```
struct RVec<T> {  
    inner: Vec<T>;  
}
```

RVec is a *wrapper* around *built-in* Vec

struct: *Refined Vectors*

```
#[refined_by(len: int)]  
struct RVec<T> {  
    inner: Vec<T>;  
}
```

refined_by: *refinement value(s)* tracked for RVec<T>

Refined Vectors: Specification

Refined Vectors: *Specification*

```
fn new() → RVec<T>[{len: 0}]
```

Create a Refined Vector

Newly *returned* vector has size 0

Refined Vectors: *Specification*

```
fn push(self: &mut RVec<T>[@v], val:T)  
  ensures  
    self: RVec<T>[{len: v.len + 1}]
```

Push value into a Refined Vector

Pushing *increases size* by 1

Refined Vectors: *Specification*

```
fn len(&RVec<T>[@v]) → usize[v.len]
```

Compute the *length* of a Refined Vector

Output `usize` indexed by *input* vector's size

Refined Vectors: *Verification*

Refined Vectors: *Verification*

```
let mut v = RVec::new(); // v: RVec<i32>[0]
```

Strong update *changes type* of `v` after each push

Refined Vectors: *Verification*

```
let mut v = RVec::new(); // v: RVec<i32>[0]  
v.push(10);              // v: RVec<i32>[1]
```

Strong update *changes type* of `v` after each push

Refined Vectors: *Verification*

```
let mut v = RVec::new(); // v: RVec<i32>[0]  
v.push(10);              // v: RVec<i32>[1]  
v.push(20);              // v: RVec<i32>[2]
```

Strong update *changes type* of *v* after each push

Refined Vectors: *Verification*

```
let mut v = RVec::new(); // v: RVec<i32>[0]
v.push(10);              // v: RVec<i32>[1]
v.push(20);              // v: RVec<i32>[2]
assert(v.len() == 2);
```

Strong update *changes type* of *v* after each push

Refined Vectors: *Verification*

```
let mut v = RVec::new(); // v: RVec<i32>[0]
v.push(10);               // v: RVec<i32>[1]
v.push(20);               // v: RVec<i32>[2]
assert(v.len() == 2);
v.push(30);               // v: RVec<i32>[3]
assert(v.len() == 2);
```

Strong update *changes type* of *v* after each push

Refined Vectors: *Verification*

```
fn init<F, A>(n: usize, mut f: F) → RVec<A>[n]  
where  
  F: FnMut(usize{v:v < n}) → A,
```

Refined Vectors: *Verification*

```
fn init<F, A>(n: usize, mut f: F) → RVec<A>[n]
where
  F: FnMut(usize{v:v < n}) → A,
{
  let mut i = 0;
  let mut res = RVec::new(); // res: RVec<i32>[0]
```

Refined Vectors: *Verification*

```
fn init<F, A>(n: usize, mut f: F) → RVec<A>[n]
where
  F: FnMut(usize{v:v < n}) → A,
{
  let mut i = 0;
  let mut res = RVec::new(); // res: RVec<i32>[0]
  while i < n {
    res.push(f(i));
    i += 1;                // res: RVec<i32>[i]
  }
}
```

Refined Vectors: *Verification*

```
fn init<F, A>(n: usize, mut f: F) → RVec<A>[n]
where
  F: FnMut(usize{v:v < n}) → A,
{
  let mut i = 0;
  let mut res = RVec::new(); // res: RVec<i32>[0]
  while i < n {
    res.push(f(i));
    i += 1;                // res: RVec<i32>[i]
  }
  res                      // res: RVec<i32>[n]
}
```

Refined Vectors: *Verification*

```
fn init<F, A>(n: usize, mut f: F) → RVec<A>[n]
where
  F: FnMut(usize{v:v < n}) → A,
{
  let mut i = 0;
  let mut res = RVec::new(); // res: RVec<i32>[0]
  while i < n {
    res.push(f(i));
    i += 1;                  // res: RVec<i32>[i]
  }
  res                        // res: RVec<i32>[n]
}
```

Refined Vectors: *Specification*

```
// get `i`-th element of vector  
fn get(&RVec<T>[@v], usize{i: i < v.len}) → &T;  
  
// set `i`-th element of vector  
fn set(&mut RVec<T>[@n], usize{i: i < v.len}, val:T);
```

Access elements of a Refined Vector

Require index i to be within vector v bounds

Refined Vectors: *Verification*

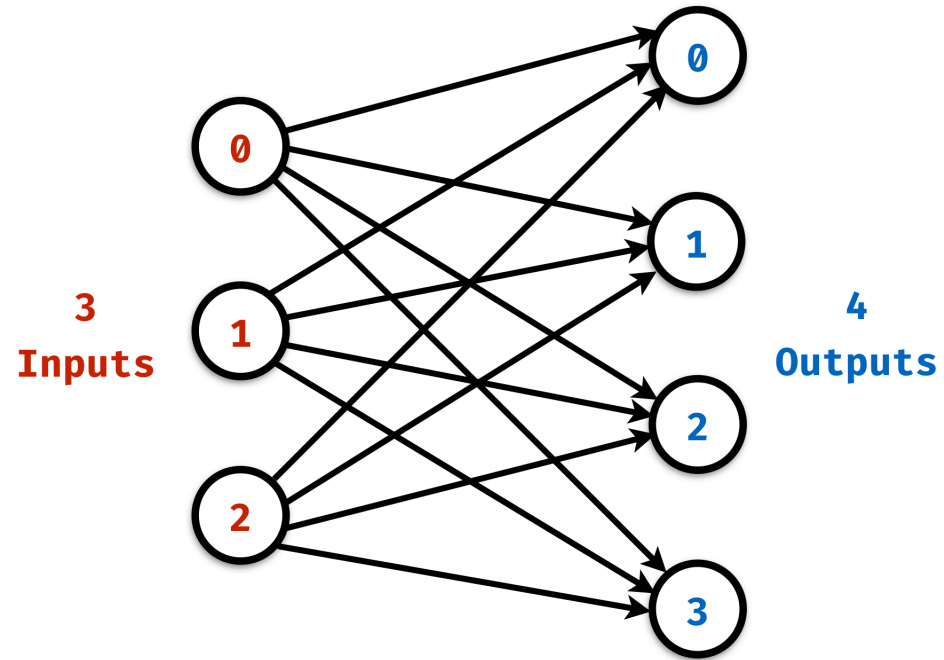
```
fn dot(x:&RVec<f64>, y:&RVec<f64>) → f64 {  
  let mut res = 0.0;  
  let mut i = 0;  
  while (i < xs.len()) {  
    res += x[i] * y[i];  
    i += 1;  
  }  
  res  
}
```

Exercise: How can we *fix* the error?

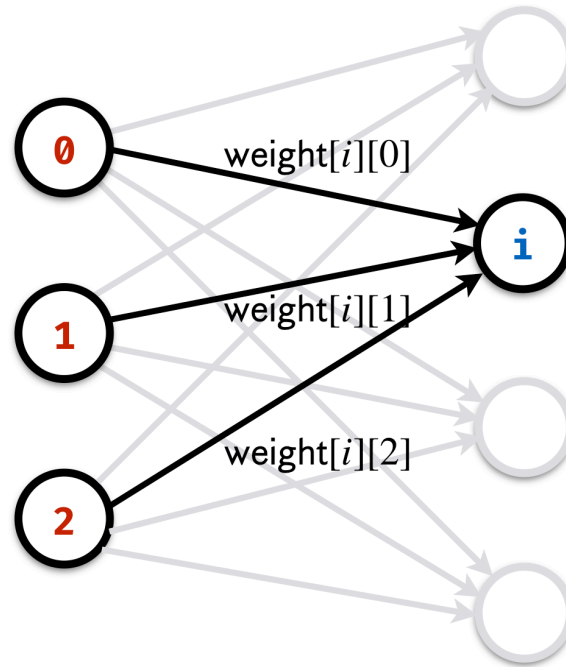
struct

Example: *Neuron Layer*

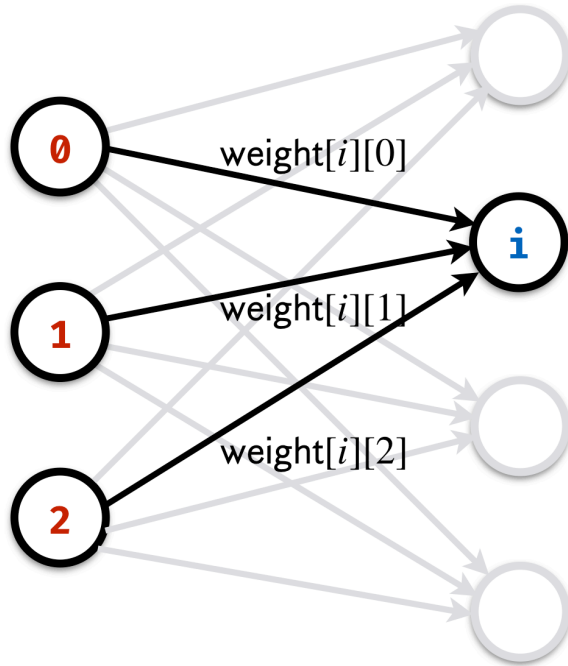
Example: *Neuron Layer*



Example: *Neuron Layer*

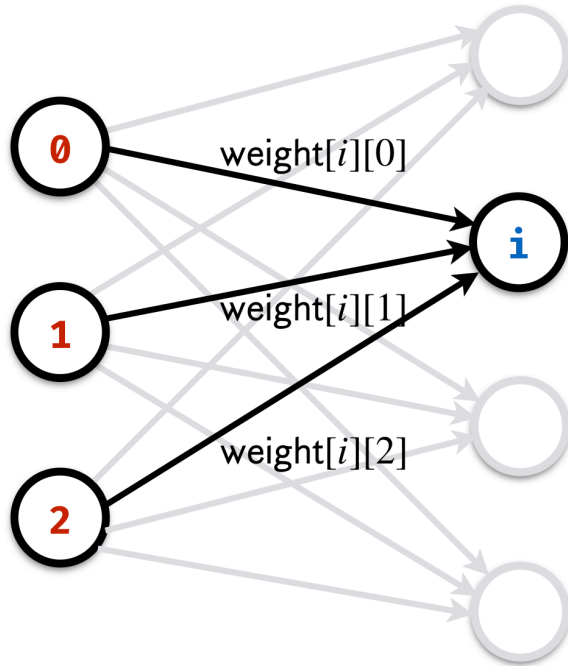


Neuron Layer: *Specification*



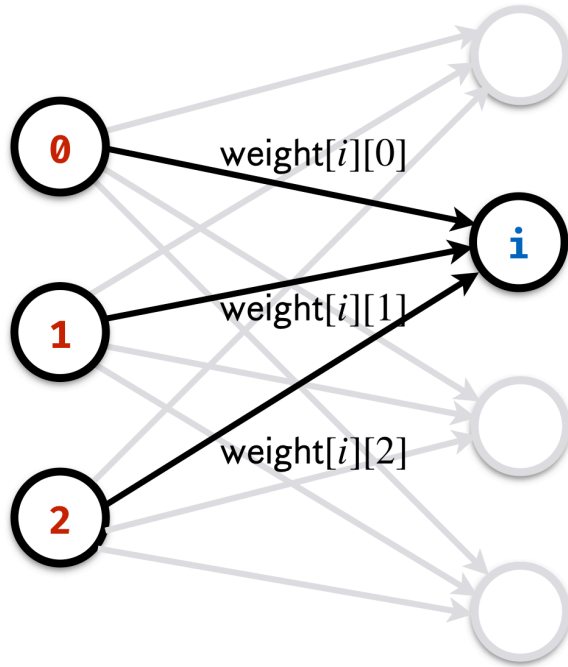
```
#[refined_by(i: int, o: int)]  
struct Layer {
```

Neuron Layer: *Specification*



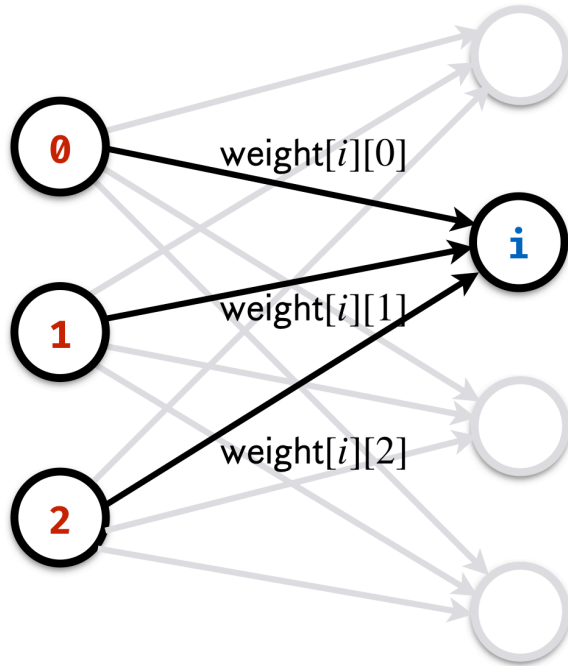
```
#[refined_by(i: int, o: int)]  
struct Layer {  
    num_inputs: usize[i],  
    num_outputs: usize[o],
```

Neuron Layer: *Specification*



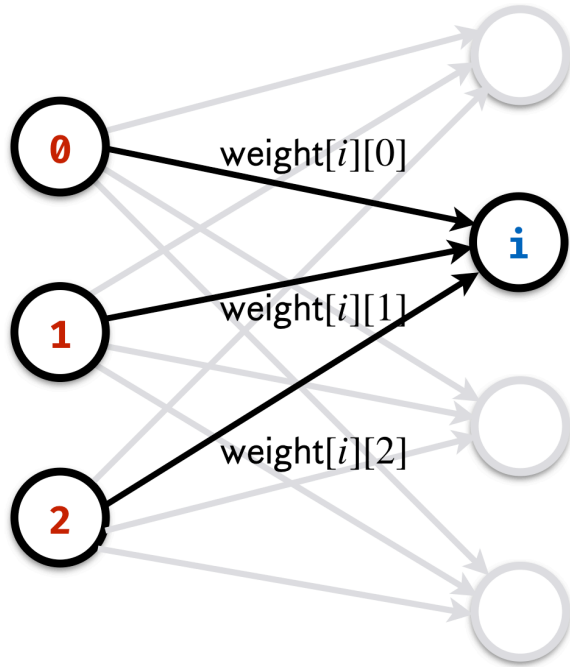
```
#[refined_by(i: int, o: int)]
struct Layer {
    num_inputs: usize[i],
    num_outputs: usize[o],
    weight: RVec<RVec<f64>[i]>[o],
```

Neuron Layer: *Specification*



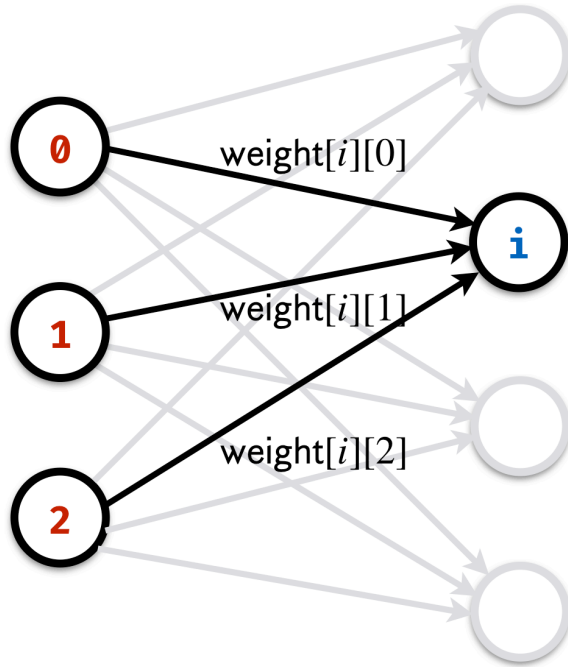
```
#[refined_by(i: int, o: int)]  
struct Layer {  
    num_inputs: usize[i],  
    num_outputs: usize[o],  
    weight: RVec<RVec<f64>[i]>[o],  
    bias: RVec<f64>[o],  
}
```

Neuron Layer: *Specification*



```
#[refined_by(i: int, o: int)]  
struct Layer {  
    num_inputs: usize[i],  
    num_outputs: usize[o],  
    weight: RVec<RVec<f64>[i]>[o],  
    bias: RVec<f64>[o],  
    outputs: RVec<f64>[o],  
}
```

Neuron Layer: *Specification*

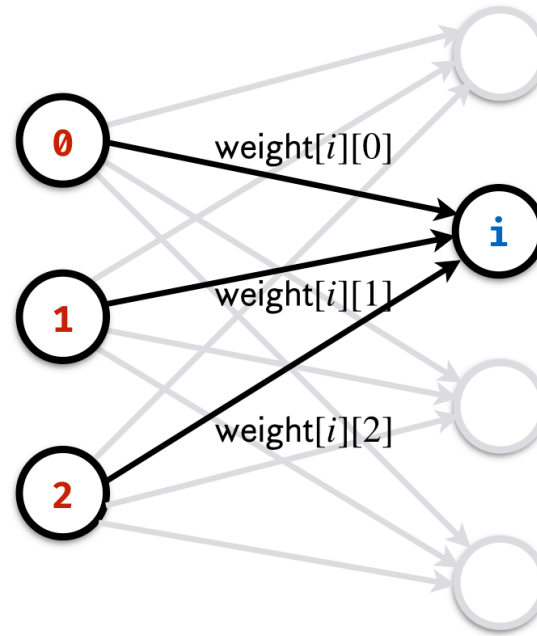


```
#[refined_by(i: int, o: int)]  
struct Layer {  
    num_inputs: usize[i],  
    num_outputs: usize[o],  
    weight: RVec<RVec<f64>[i]>[o],  
    bias: RVec<f64>[o],  
    outputs: RVec<f64>[o],  
}
```

Neuron Layer: *Verification*

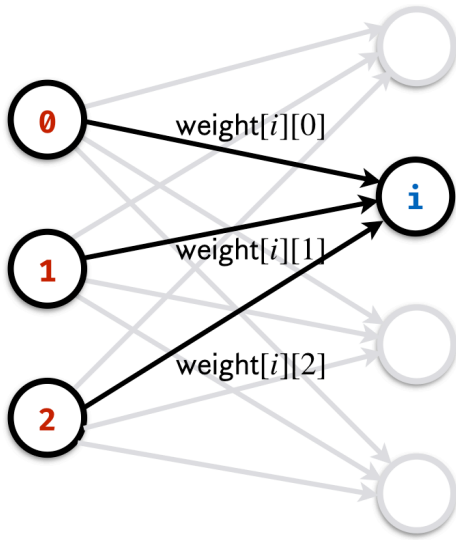
```
fn new(i: usize, o: usize) → Layer[i, o] {  
    let mut rng = rand::thread_rng();  
    Layer {  
        num_inputs: i,  
        num_outputs: o,  
        weight: init(o, |_|  
                     init(i, |_|  
                          rng.gen_range(-1.0..1.0))),  
        bias: init(o, |_| rng.gen_range(-1.0..1.0)),  
        outputs: init(o, |_| 0.0),  
    }  
}
```

Neuron Layer: *Forward Propagation*



$$\text{out}[i] \doteq \sigma(\text{in} \cdot \text{weight}[i] + \text{bias}[i])$$

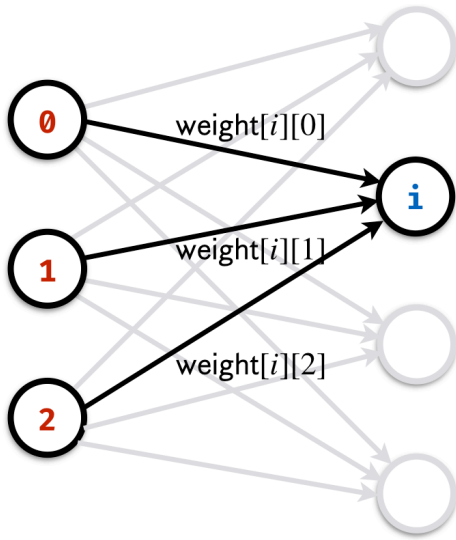
Neuron Layer: *Forward Propagation*



$$\text{out}[i] \doteq \sigma(\text{in} \cdot \text{weight}[i] + \text{bias}[i])$$

```
fn forward(&mut self, input: &RVec<f64>) {
```

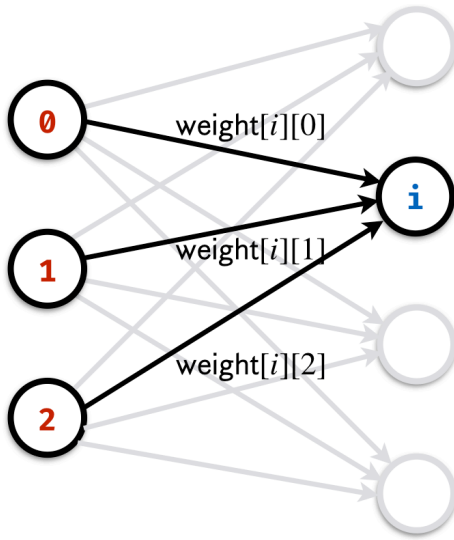
Neuron Layer: *Forward Propagation*



$$\text{out}[i] \doteq \sigma(\text{in} \cdot \text{weight}[i] + \text{bias}[i])$$

```
fn forward(&mut self, input: &RVec<f64>) {  
    (0..self.num_outputs).for_each(|i| {
```

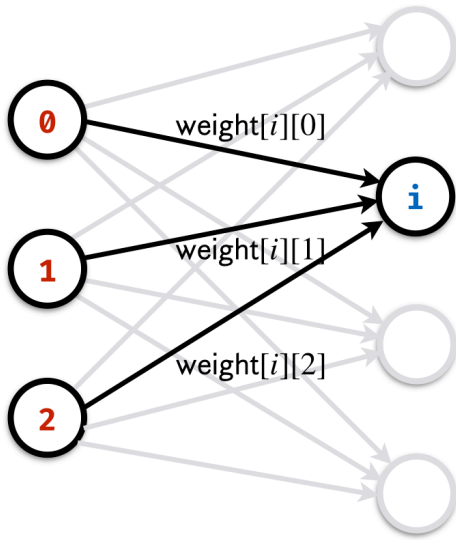
Neuron Layer: *Forward Propagation*



```
fn forward(&mut self, input: &RVec<f64>) {  
    (0..self.num_outputs).for_each(|i| {  
        let wt = dot(&self.weight[i], input);
```

$$\text{out}[i] \doteq \sigma(\text{in} \cdot \text{weight}[i] + \text{bias}[i])$$

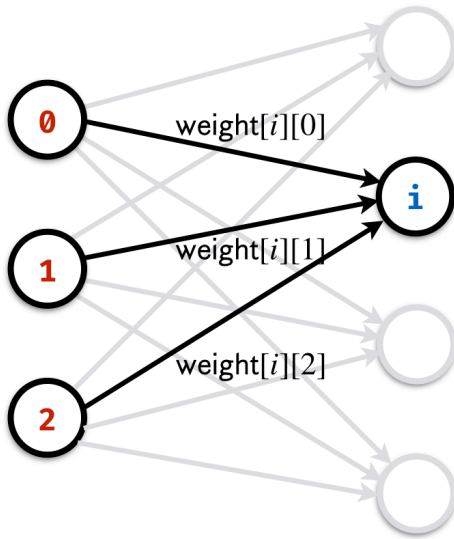
Neuron Layer: *Forward Propagation*



```
fn forward(&mut self, input: &RVec<f64>) {  
    (0..self.num_outputs).for_each(|i| {  
        let wt = dot(&self.weight[i], input);  
        let sum = wt + self.bias[i];  
    })  
}
```

$$\text{out}[i] \doteq \sigma(\text{in} \cdot \text{weight}[i] + \text{bias}[i])$$

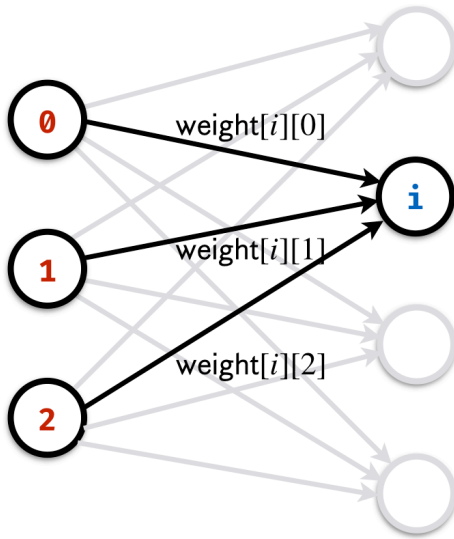
Neuron Layer: *Forward Propagation*



```
fn forward(&mut self, input: &RVec<f64>) {  
    (0..self.num_outputs).for_each(|i| {  
        let wt = dot(&self.weight[i], input);  
        let sum = wt + self.bias[i];  
        self.outputs[i] = sigmoid(sum);  
    })  
}
```

$$\text{out}[i] \doteq \sigma(\text{in} \cdot \text{weight}[i] + \text{bias}[i])$$

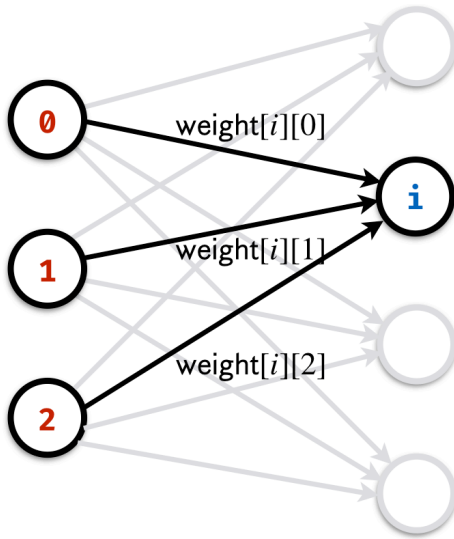
Neuron Layer: *Forward Propagation*



$$\text{out}[i] \doteq \sigma(\text{in} \cdot \text{weight}[i] + \text{bias}[i])$$

```
fn forward(&mut self, input: &RVec<f64>) {  
    (0..self.num_outputs).for_each(|i| {  
        let wt = dot(&self.weight[i], input);  
        let sum = wt + self.bias[i];  
        self.outputs[i] = sigmoid(sum);  
    })  
}
```

Neuron Layer: *Forward Propagation*



$$\text{out}[i] \doteq \sigma(\text{in} \cdot \text{weight}[i] + \text{bias}[i])$$

```
fn forward(&mut self, input: &RVec<f64>) {  
    (0..self.num_outputs).for_each(|i| {  
        let wt = dot(&self.weight[i], input);  
        let sum = wt + self.bias[i];  
        self.outputs[i] = sigmoid(sum);  
    })  
}
```

Exercise: Can you *fix* the error?

2. *Types*

struct

“Product” types

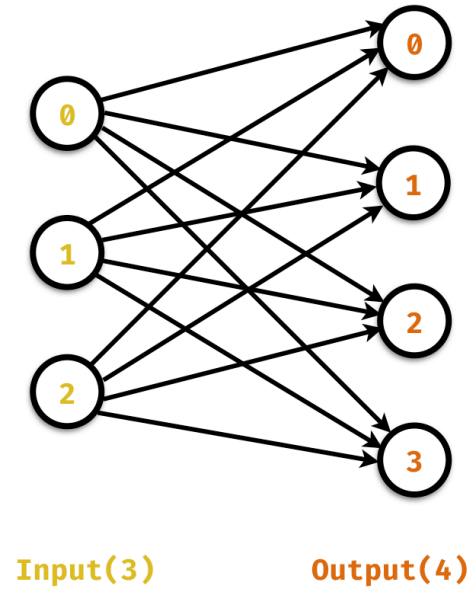
enum

“Sum” types

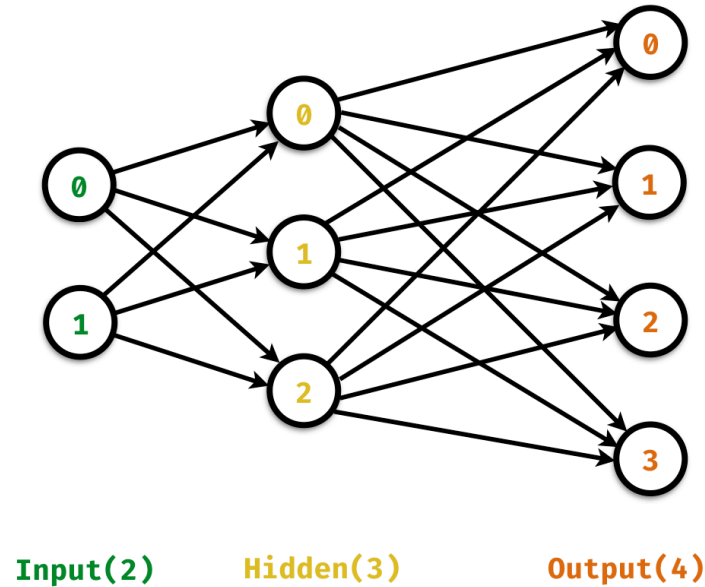
enum

Example: *Neural Network*

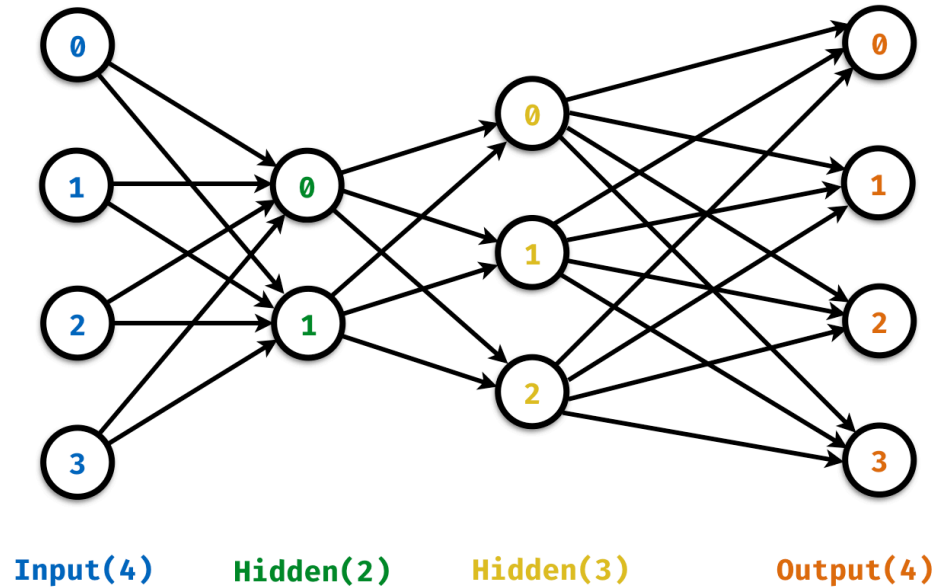
Neural Network has *Many* Layers



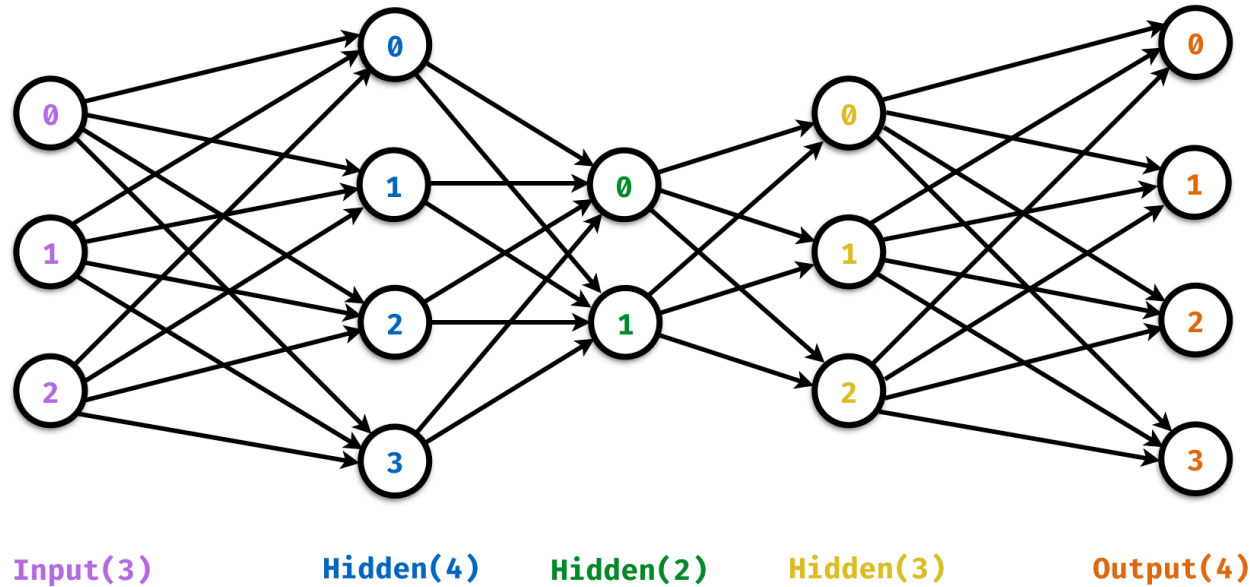
Neural Network has *Many* Layers



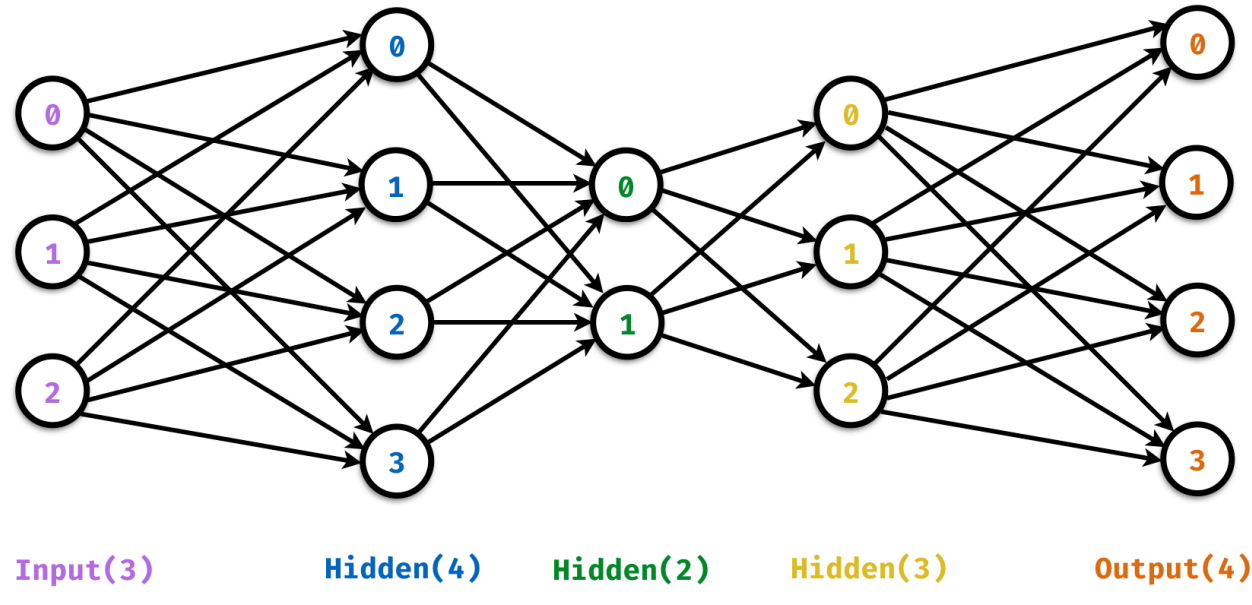
Neural Network has *Many* Layers



Neural Network has *Many* Layers



Neural Network has *Many* Layers



How to ensure layers *compose* correctly?

Neural Network: *Specification*

```
enum Network {  
    Last(Layer),  
    Next(Layer, Box<Network>),  
}
```

How to ensure layers *compose* correctly?

Neural Network: *Specification*

```
#[refined_by(i: int, o: int)]  
enum Network {
```

How to ensure layers *compose* correctly?

Neural Network: *Specification*

```
#[refined_by(i: int, o: int)]  
enum Network {  
    Last(Layer[@i, @o]) → Network[i, o],
```

How to ensure layers *compose* correctly?

Neural Network: *Specification*

```
#[refined_by(i: int, o: int)]  
enum Network {  
    Last(Layer[@i, @o]) → Network[i, o],  
    Next(Layer[@i, @n], Box<Network[n, @o]>) → Network[i, o],  
}
```

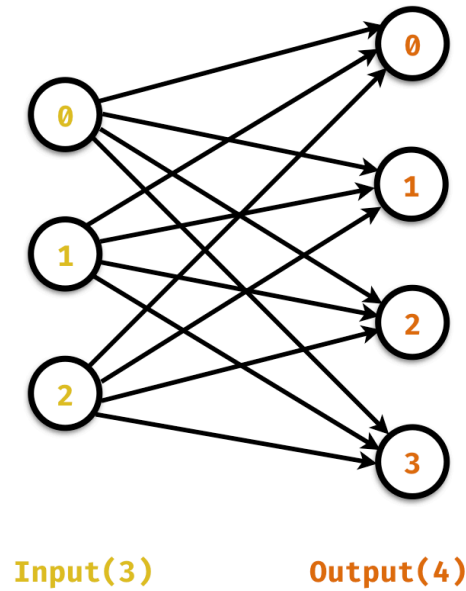
How to ensure layers *compose* correctly?

Neural Network: *Specification*

```
#[refined_by(i: int, o: int)]  
enum Network {  
    Last(Layer[in, out]) → Network[i, o],  
    Next(Layer[in, hidden], Box<Network[hidden, out>) → Network[i, o],  
}
```

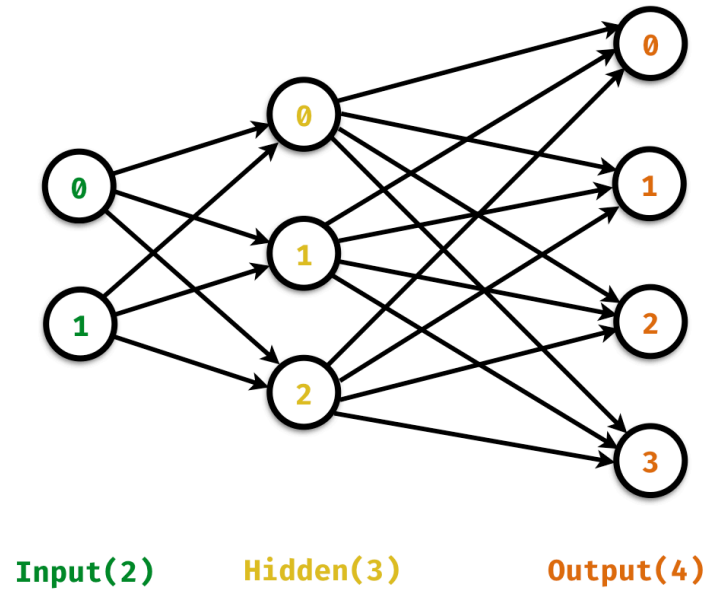
How to ensure layers *compose* correctly?

Refinements Ensure *Correct Composition*



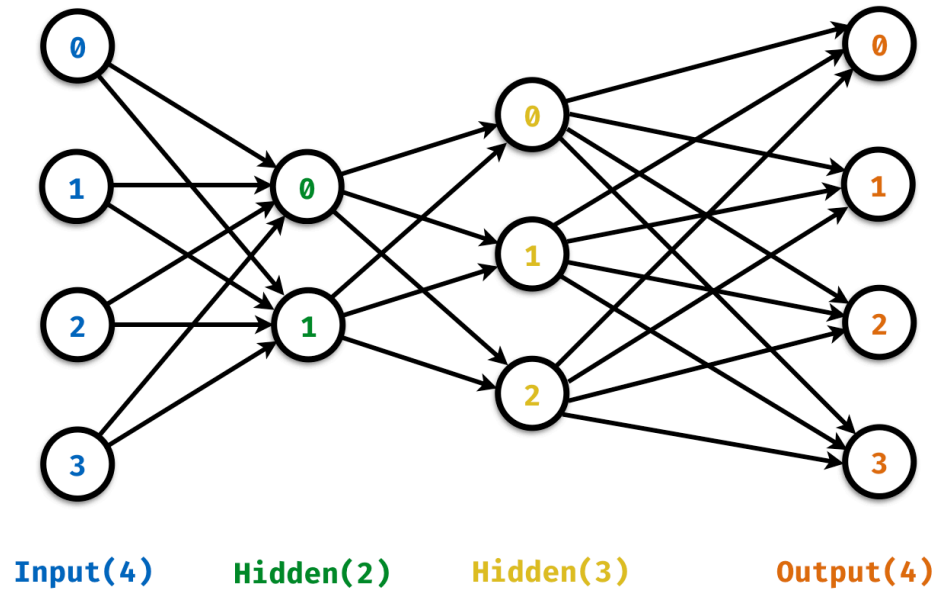
`Last(Layer[3,4])` $\longrightarrow$ `Network[3, 4]`

Refinements Ensure Correct *Composition*



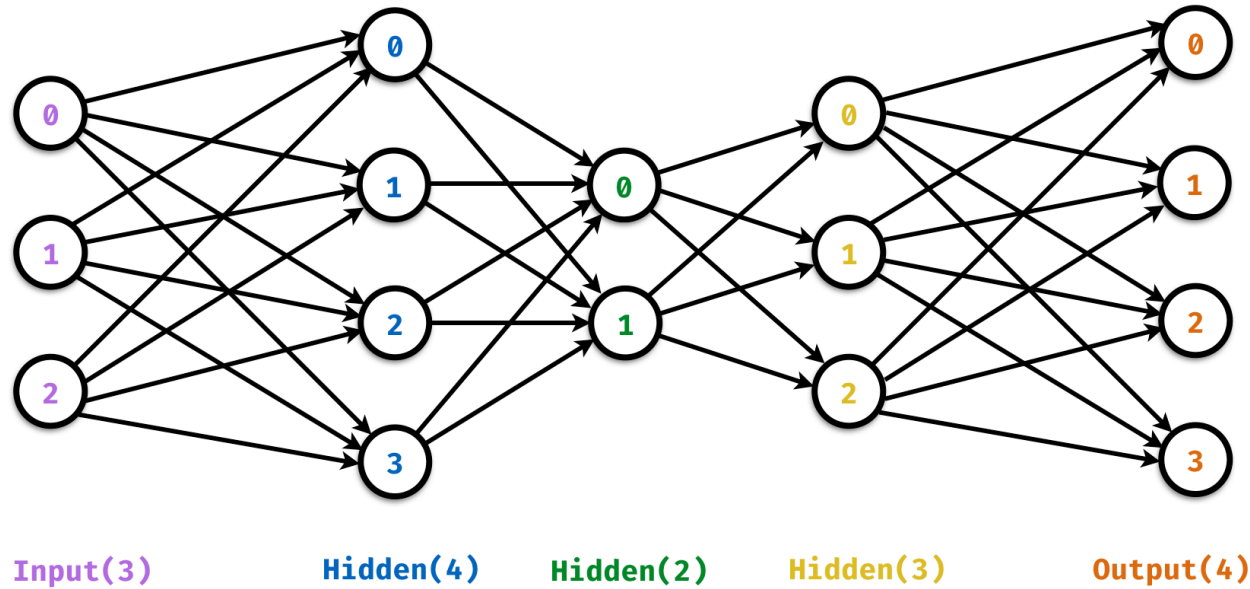
`Next(Layer[2,3], Network[3,4])  $\longrightarrow$  Network[2, 4]`

Refinements Ensure Correct *Composition*



`Next(Layer[4,2], Network[2,4])  $\longrightarrow$  Network[4, 4]`

Refinements Ensure Correct *Composition*



`Next(Layer[3,4], Network[4,4])  $\longrightarrow$  Network[4, 4]`

Neural Network: *Verification*

```
fn new(input: usize, hidden: &[usize], output: usize)  
    → Network[input, output]
```

Neural Network: *Verification*

```
fn new(input: usize, hidden: &[usize], output: usize)
    → Network[input, output]
{
    if hidden_sizes.len() == 0 {
        Network::Last(Layer::new(input, output))
    }
}
```

Neural Network: *Verification*

```
fn new(input: usize, hidden: &[usize], output: usize)
    → Network[input, output]
{
    if hidden_sizes.len() == 0 {
        Network::Last(Layer::new(input, output))
    } else {
        let n = hidden[0];
```

Neural Network: *Verification*

```
fn new(input: usize, hidden: &[usize], output: usize)
    → Network[input, output]
{
    if hidden_sizes.len() == 0 {
        Network::Last(Layer::new(input, output))
    } else {
        let n = hidden[0];
        let layer = Layer::new(input, n);
```

Neural Network: *Verification*

```
fn new(input: usize, hidden: &[usize], output: usize)
    → Network[input, output]
{
    if hidden_sizes.len() == 0 {
        Network::Last(Layer::new(input, output))
    } else {
        let n = hidden[0];
        let layer = Layer::new(input, n);
        let rest = Network::new(n, &hidden[1..], output);
```

Neural Network: *Verification*

```
fn new(input: usize, hidden: &[usize], output: usize)
    → Network[input, output]
{
    if hidden_sizes.len() == 0 {
        Network::Last(Layer::new(input, output))
    } else {
        let n = hidden[0];
        let layer = Layer::new(input, n);
        let rest = Network::new(n, &hidden[1..], output);
        Network::Next(layer, Box::new(rest))
    }
}
```

Neural Network: *Verification*

```
fn new(input: usize, hidden: &[usize], output: usize)
    → Network[input, output]
{
    if hidden_sizes.len() == 0 {
        Network::Last(Layer::new(input, output))
    } else {
        let n = hidden[0];
        let layer = Layer::new(input, n);
        let rest = Network::new(n, &hidden[1..], output);
        Network::Next(layer, Box::new(rest))
    }
}
```

Neural Network: *Verification*

```
fn forward(&mut Network, input: &RVec<f64>) → RVec<f64> {
```

Neural Network: *Verification*

```
fn forward(&mut Network, input: &RVec<f64>) → RVec<f64> {  
  match self {  
    NeuralNetwork::Last(layer) ⇒ {
```

Neural Network: *Verification*

```
fn forward(&mut Network, input: &RVec<f64>) → RVec<f64> {  
  match self {  
    NeuralNetwork::Last(layer) ⇒ {  
      layer.forward(input);  
      layer.outputs.clone()  
    }  
  }  
}
```

Neural Network: *Verification*

```
fn forward(&mut Network, input: &RVec<f64>) → RVec<f64> {  
  match self {  
    NeuralNetwork::Last(layer) ⇒ {  
      layer.forward(input);  
      layer.outputs.clone()  
    }  
    NeuralNetwork::Next(layer, next) ⇒ {
```

Neural Network: *Verification*

```
fn forward(&mut Network, input: &RVec<f64>) → RVec<f64> {  
  match self {  
    NeuralNetwork::Last(layer) ⇒ {  
      layer.forward(input);  
      layer.outputs.clone()  
    }  
    NeuralNetwork::Next(layer, next) ⇒ {  
      layer.forward(input);  
    }  
  }  
}
```

Neural Network: *Verification*

```
fn forward(&mut Network, input: &RVec<f64>) → RVec<f64> {  
    match self {  
        NeuralNetwork::Last(layer) ⇒ {  
            layer.forward(input);  
            layer.outputs.clone()  
        }  
        NeuralNetwork::Next(layer, next) ⇒ {  
            layer.forward(input);  
            next.forward(&layer.outputs)  
        }  
    }  
}
```

Neural Network: *Verification*

```
fn forward(&mut Network, input: &RVec<f64>) → RVec<f64> {  
    match self {  
        NeuralNetwork::Last(layer) ⇒ {  
            layer.forward(input);  
            layer.outputs.clone()  
        }  
        NeuralNetwork::Next(layer, next) ⇒ {  
            layer.forward(input);  
            next.forward(&layer.outputs)  
        }  
    }  
}
```

Neural Network: *Verification*

```
fn forward(&mut Network, input: &RVec<f64>) → RVec<f64> {  
  match self {  
    NeuralNetwork::Last(layer) ⇒ {  
      layer.forward(input);  
      layer.outputs.clone()  
    }  
    NeuralNetwork::Next(layer, next) ⇒ {  
      layer.forward(input);  
      next.forward(&layer.outputs)  
    }  
  }  
}
```

Exercise: *Fix* the specification for forward?

2. *Types*

Compositional specification & verification

struct

enum

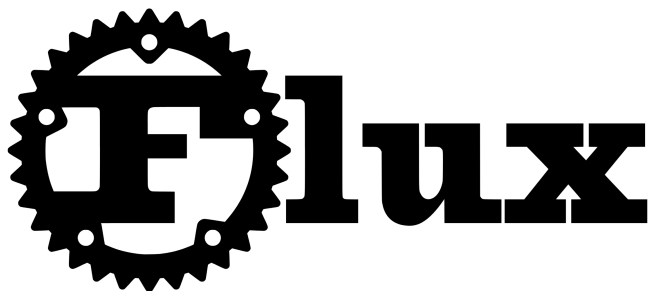
2. *Types*

Compositional specification & verification

struct

enum

“Make illegal states unrepresentable”



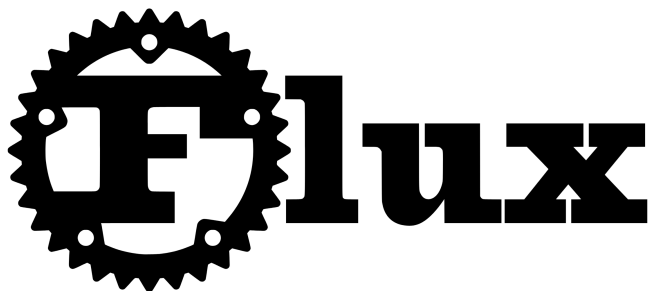
1. Refinement

Index, Exist & Update

2. Types

Struct & Enum

3. Verified Systems



1. Refinement

Index, Exist & Update

2. Types

Struct & Enum

3. Verified Systems

3. Verified Systems

Verified Systems

Tok

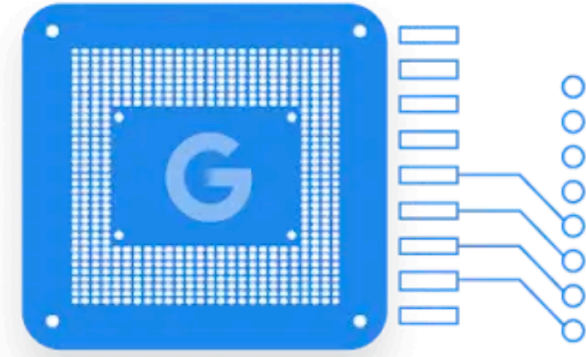
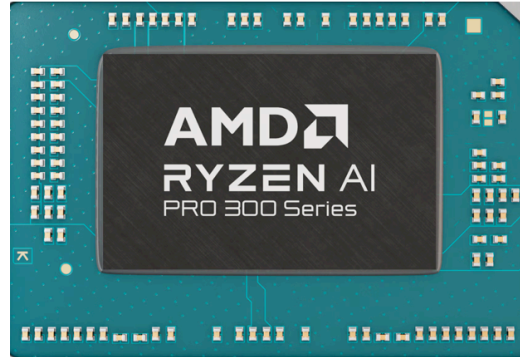
Verified Systems



An Embedded OS Kernel

Levy *et al.* SOSP 2017, Schuermann *et al.* SOSP 2025

Verified Systems



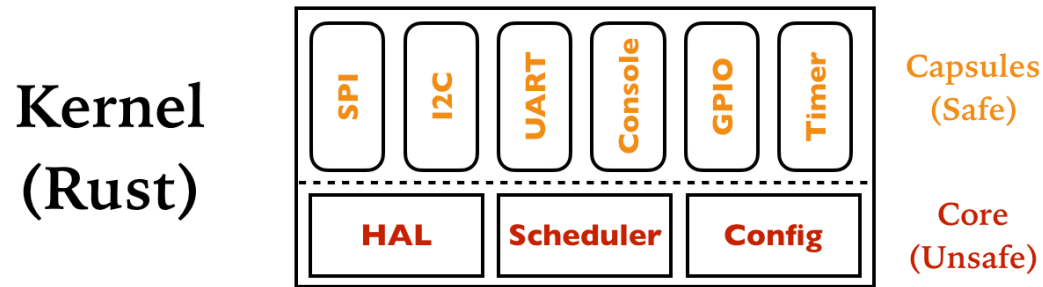
An Embedded OS Kernel

Firmware in Microsoft Pluton “root of trust”, Google ChromeBooks (GSC),...

Tock: *Architecture*

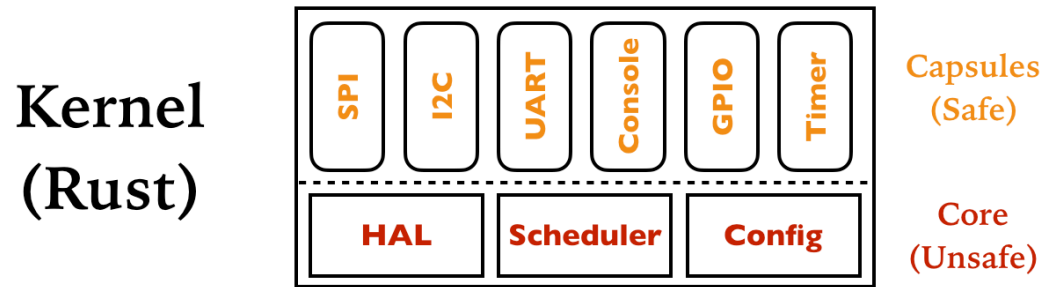
Tock: *Architecture*

Tock: Architecture



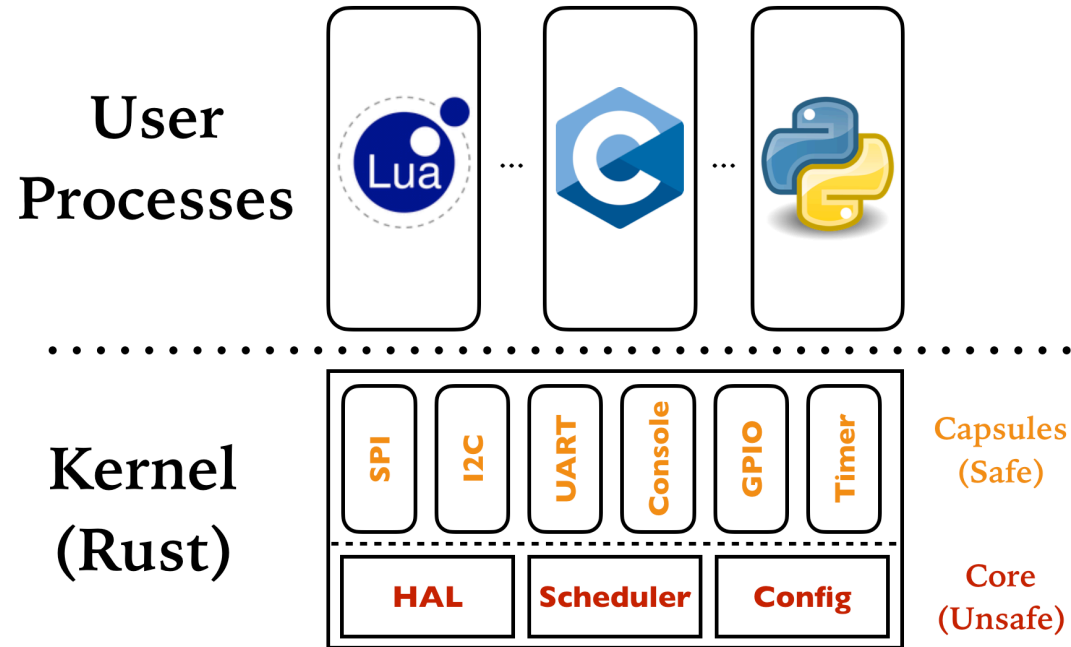
Capsules enjoy Rust's safety guarantees

Tock: Architecture



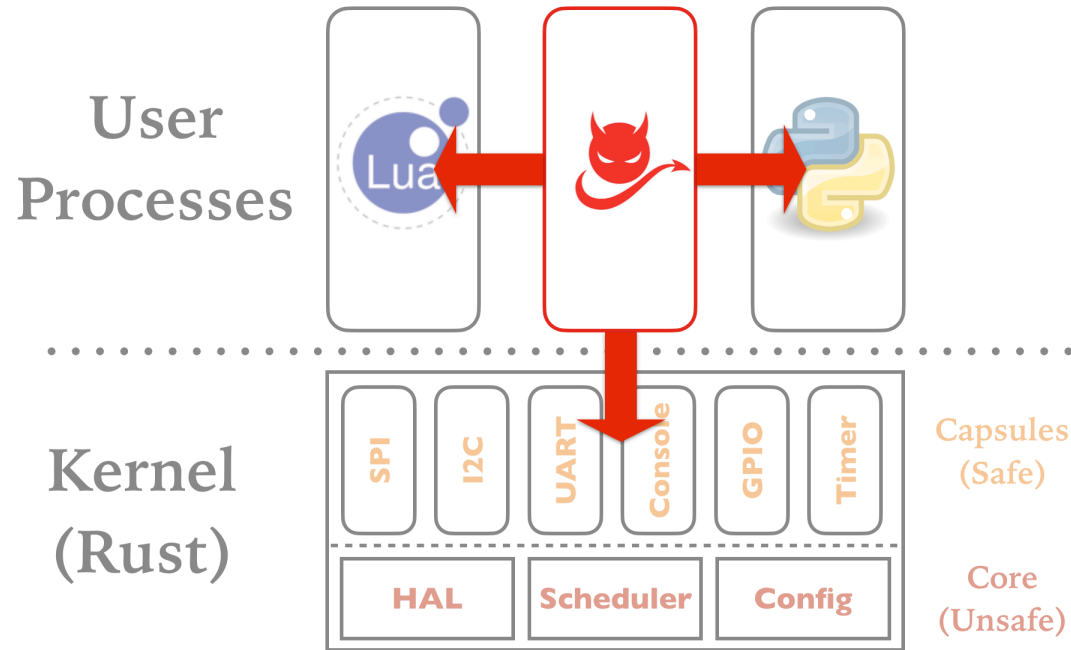
Core uses unsafe to manage hardware

Tock: Architecture



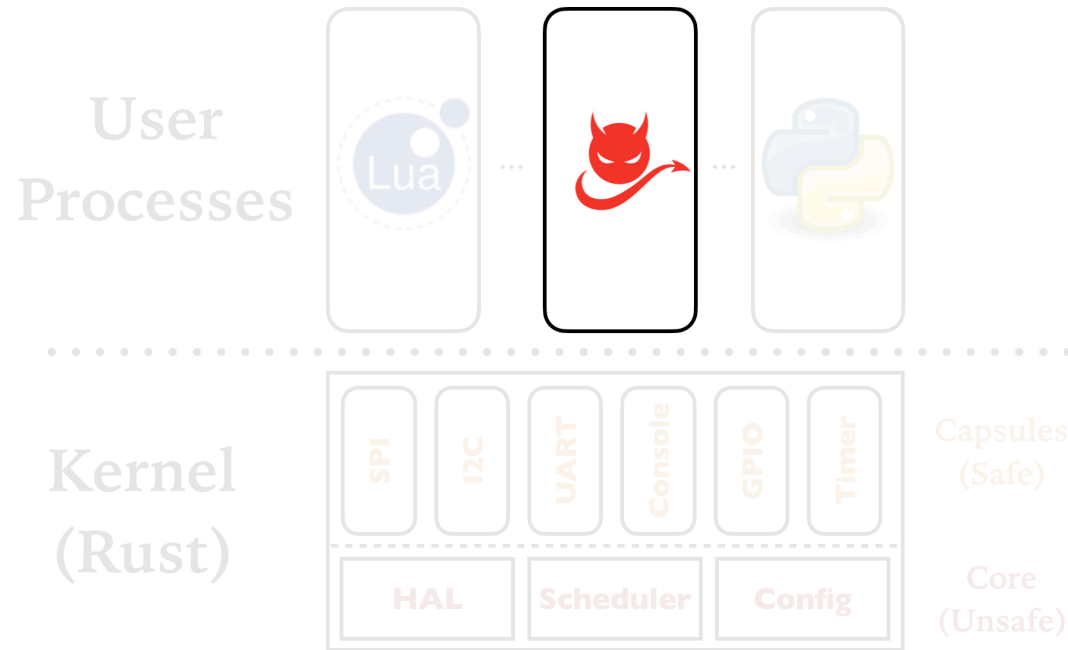
User processes can run *arbitrary code*

Tock: Architecture



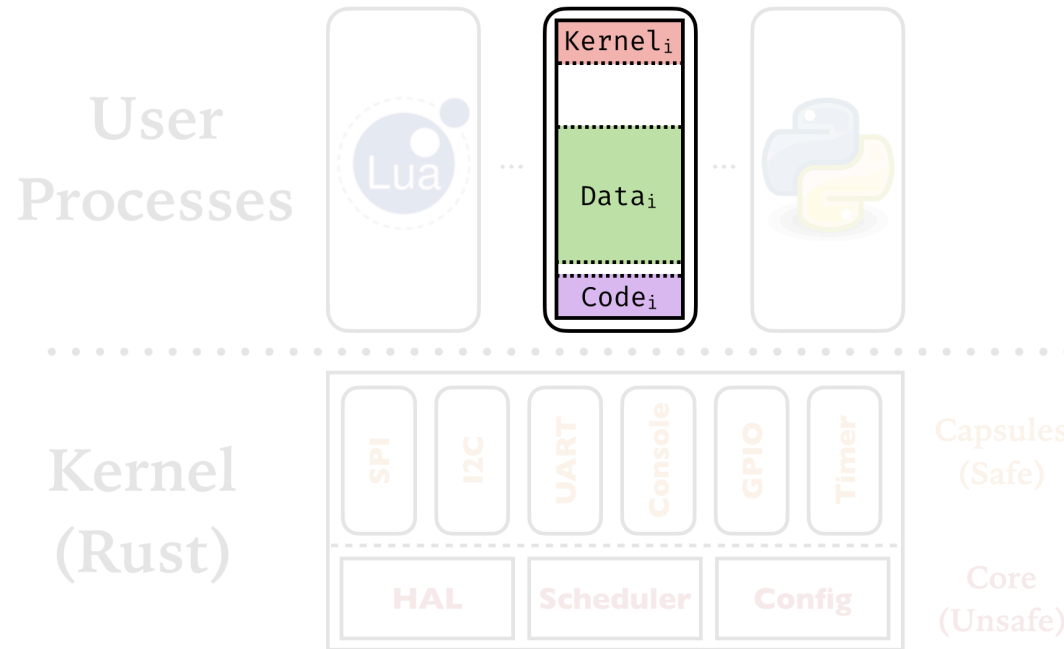
User processes run *arbitrary code* with vulnerabilities

Tock: Security by *Isolation*



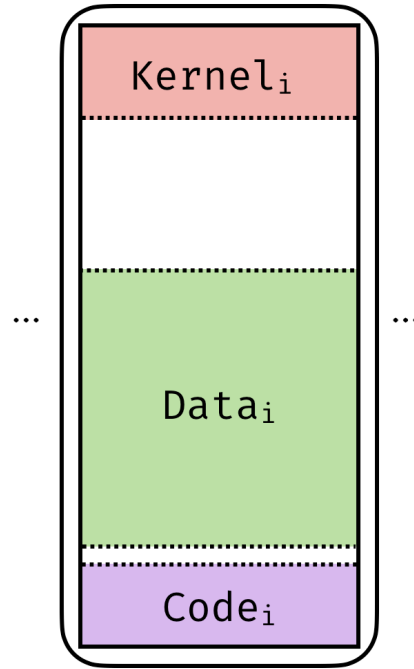
Isolate processes from kernel and each other

Tock: Security by *Isolation*



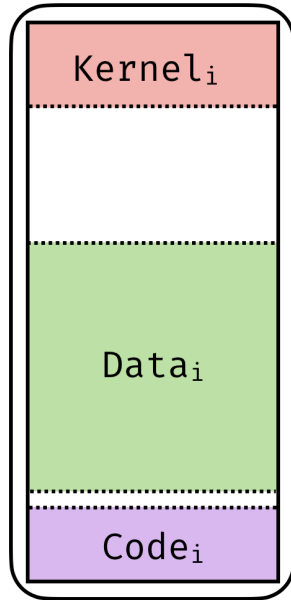
Process Memory: `Kernel` + `Data` + `Code`

Tock: Security by *Isolation*



Process Memory: $\text{Kernel} + \text{Data} + \text{Code}$

Tock: Security by *Isolation*



Ensure that **Proc_i**

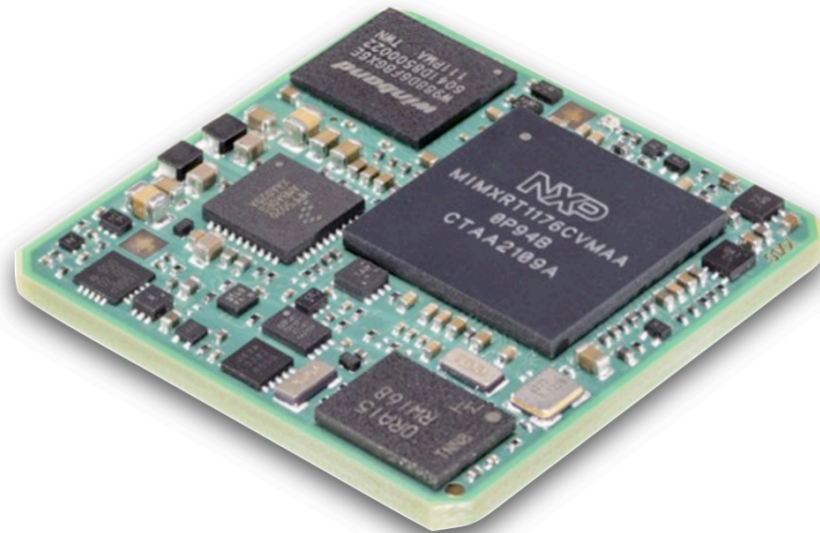
Cannot access Kernel_i

Can only Rd+Wr in Data_i

Can only Rd+Ex in Code_i

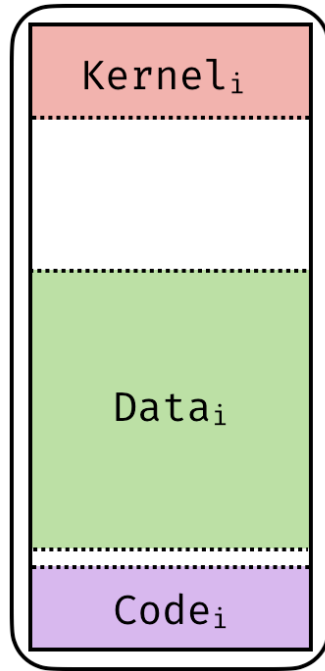
Process Memory: $\text{Kernel} + \text{Data} + \text{Code}$

Isolation via Memory Protection Units

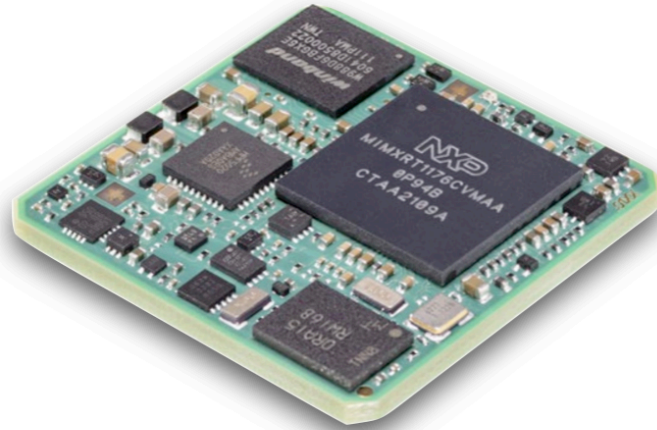
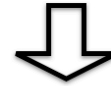


MPU controls address (regions) allowed to Rd, Wr, Ex

Isolation via Memory Protection Units



Configure MPU at switch to **Proc_i**



To *only* allow Rd+Wr in Data_i and Rd+Ex in Code_i

Isolation via Memory Protection Units

What could *possibly* go wrong?

Isolation via Memory Protection Units

What could *possibly* go wrong?

rv32i/{pmp,epmp}: invalidate unused regions on config switch #3541

 Merged bradjc merged 1 commit into [tock:master](#) from [lschuermann:otdev/pmp-invalidate-fix](#) on Jul 18, 2023

arch: cortex: fix MPU rounding calculation #2435

 Merged bors merged 2 commits into [master](#) from [cortex-mpu-rounding-calculation](#) on Feb 24, 2021

kernel: process: reconfigure mpu on restart #2146

 Merged bors merged 2 commits into [master](#) from [kernel-process-restart-mpu](#) on Oct 23, 2020

Potential sharing of kernel memory #1141

 Closed  #1144

Cortex-M MPU: [allocate_app_memory_region](#) allows access to kernel grant memory #4366

 Closed  #4384

riscv: pmp: disallow access above app brk #2173

 Merged bradjc merged 1 commit into [master](#) from [pmp-exclude-grant-region](#) on Oct 23, 2020

arch/rv32i: pmp/ePMP: Fixup PMP comparision #2947

 Merged bors merged 2 commits into [tock:master](#) from [alistair23:alistair/pmp-fixup](#) on Jan 28, 2022

rv32: pmp: Fix bounds #2076

 Merged bors merged 1 commit into [master](#) from [pmp-address-fix](#) on Aug 19, 2020

What could *possibly* go wrong?

Buggy *MPU configuration*

What could *possibly* go wrong?

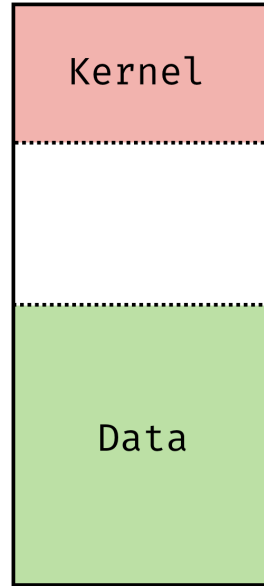
Buggy *MPU configuration*

Buggy *interrupts & context switching*

Buggy *MPU Configuration*

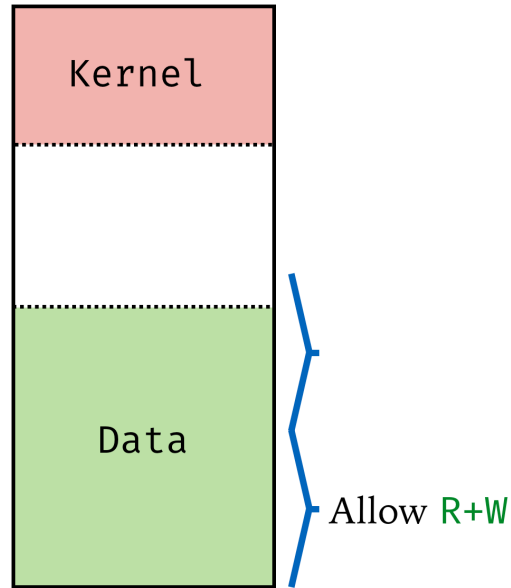
MPUs permit access control over *fixed size regions*

Buggy *MPU Configuration*



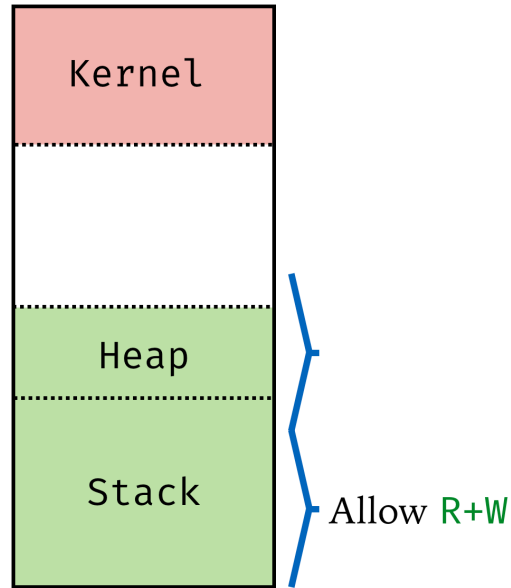
MPUs permit access control over *fixed size regions*

Buggy MPU Configuration



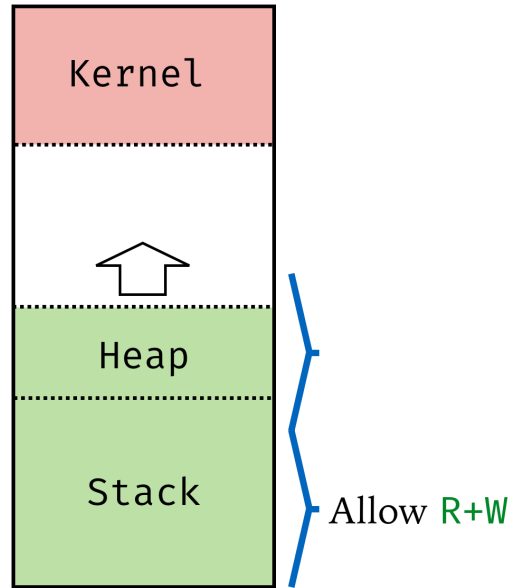
MPUs permit access control over *fixed size regions*

Buggy *MPU Configuration*



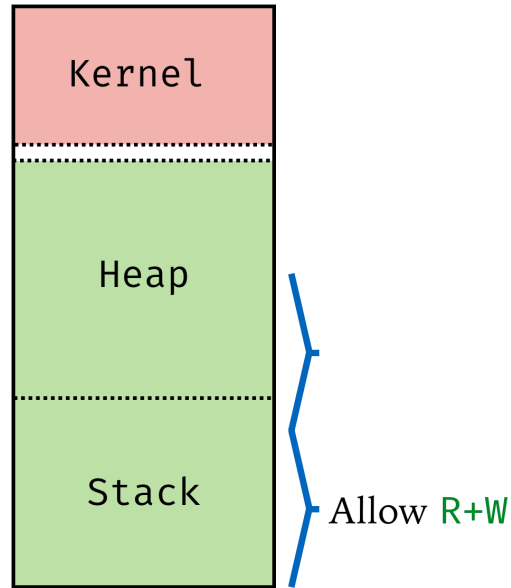
Process Accesses Heap + Stack

Buggy *MPU Configuration*



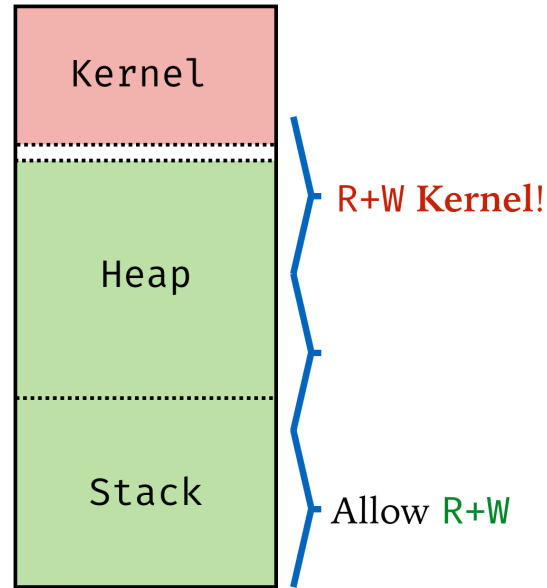
Heap can *grow* via kernel syscall

Buggy *MPU Configuration*



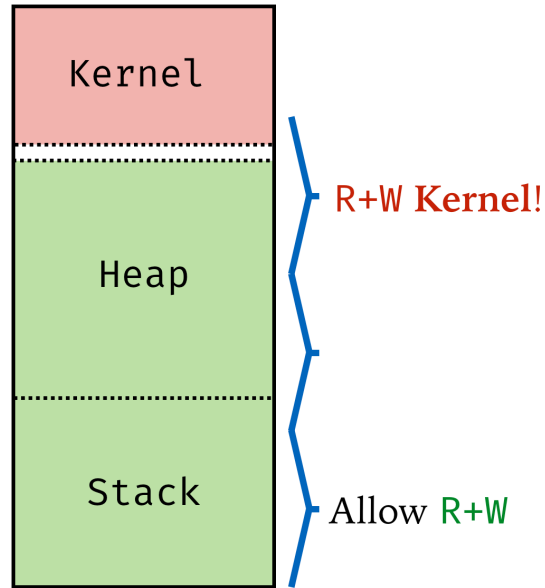
Heap can *grow* via kernel syscall

Buggy MPU Configuration



Covering *regions* lets process R+W kernel data!

Buggy *MPU* Configuration



Challenge: Ensure *gap* to avoid overlap!

Verified MPU Configuration

Challenge: Ensure *gap* to avoid overlap!

Verified MPU Configuration

Challenge: Ensure *gap* to avoid overlap!

Refine kernel's type for *process memory*

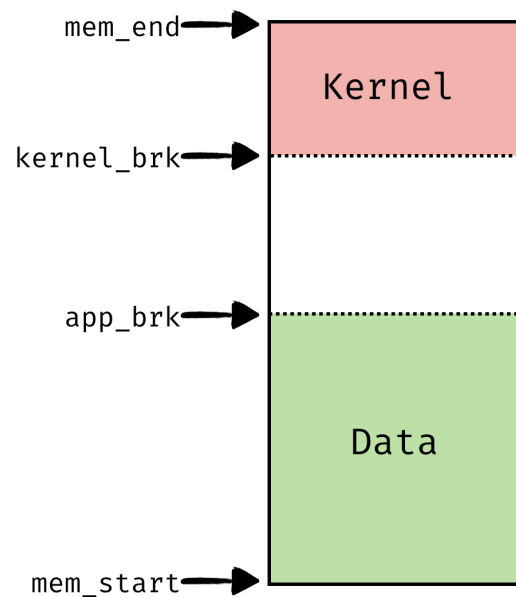
Verified MPU Configuration

Challenge: Ensure *gap* to avoid overlap!

Refine kernel's type for *process memory*

Refine MPU's type for *region configuration*

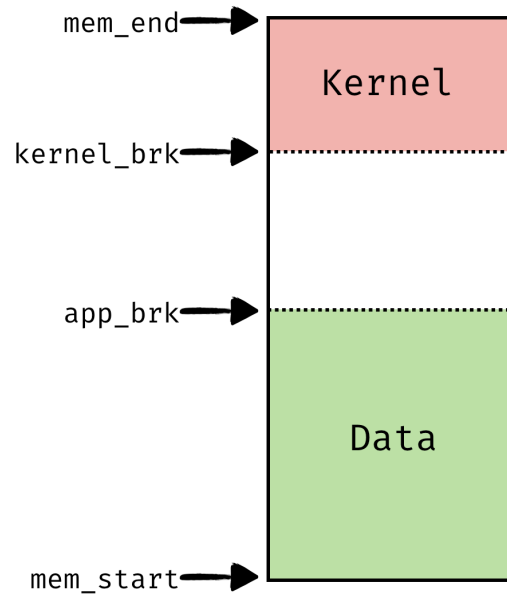
Verified MPU Configuration



Refine kernel's type for *process memory*

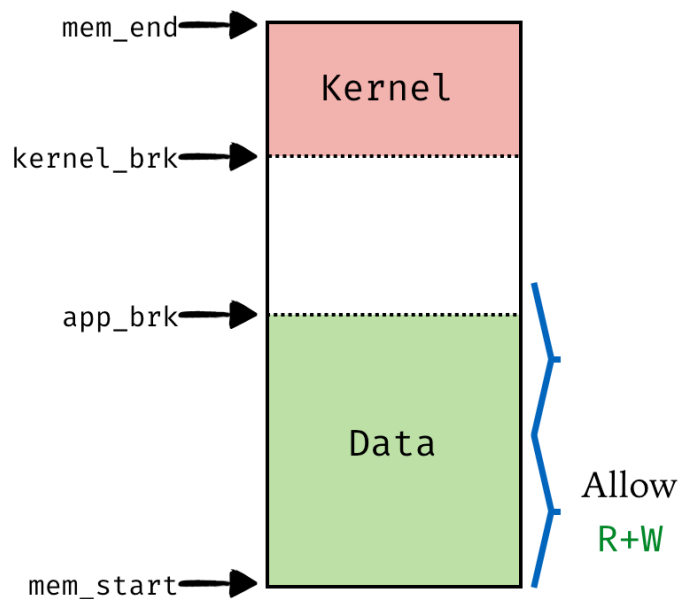
Verified MPU Configuration

```
struct ProcessMemory
{
  mem_end: Ptr[v: mem_start < v],
  kernel_brk: Ptr[v: app_brk < v],
  app_brk: Ptr[v: mem_start <= v],
  mem_start: Ptr,
}
```



Refine kernel's type for *process memory*

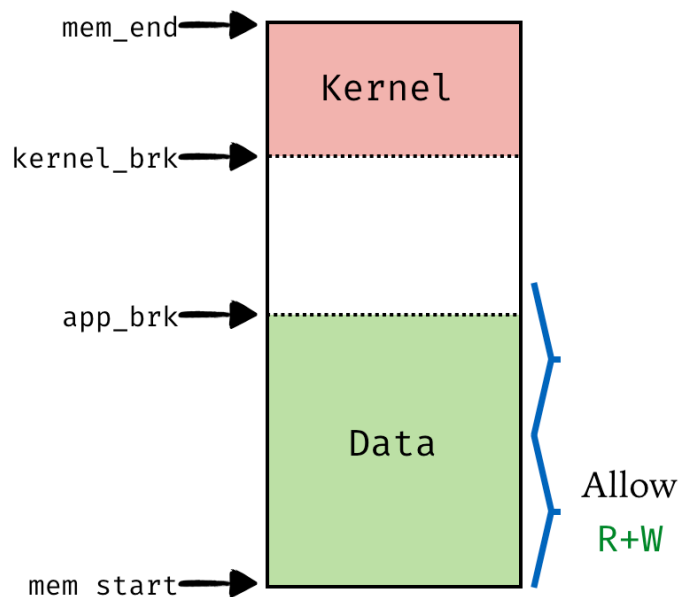
Verified MPU Configuration



```
struct RegionConfig<R: Region>,  
{  
    brks: Breaks,  
    regions: Array<R>{v: ok_regions(brks, v)}  
}
```

Refine MPU's type for *region configuration*

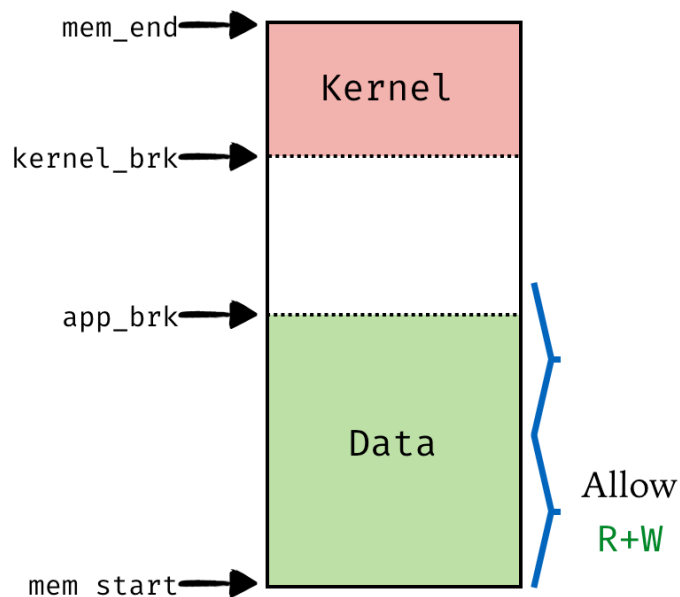
Verified MPU Configuration



```
struct RegionConfig<R: Region>,  
{  
    brks: Breaks,  
    regions: Array<R>{v: ok_regions(brks, v)}  
}  
  
fn ok_regions(brks, regions) → Bool {  
    can_access_ram(breaks, regions) &&  
    cannot_access_other(breaks, regions)  
}
```

Refine MPU's type for *region configuration*

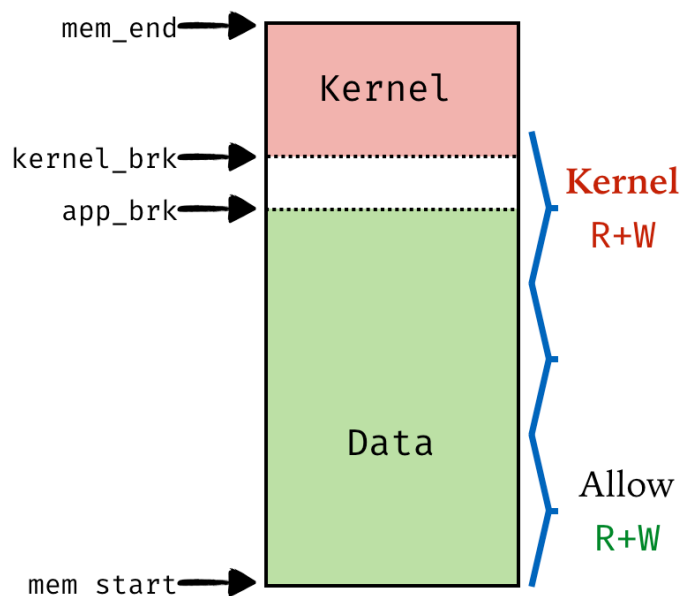
Verified MPU Configuration



```
struct RegionConfig<R: Region>,  
{  
    brks: Breaks,  
    regions: Array<R>{v: ok_regions(brks, v)}  
}  
  
fn ok_regions(brks, regions) → Bool {  
    can_access_ram(breaks, regions) &&  
    cannot_access_other(breaks, regions)  
}
```

Refine MPU's type for *region configuration*

Verified MPU Configuration



```
struct RegionConfig<R: Region>,  
{  
    brks: Breaks,  
    regions: Array<R>{v: ok_regions(brks, v)}  
}  
  
fn ok_regions(brks, regions) → Bool {  
    can_access_ram(breaks, regions) &&  
    cannot_access_other(breaks, regions)  
}
```

Refine MPU's type for *region configuration*

Verified *Isolation* in Tock [SOSP'25]

Verified *MPU configuration*

Verified *interrupts & context switching*

Verified *Isolation* in Tock [SOSP'25]

Verified *MPU configuration*

Verified *interrupts & context switching*

Verified Interrupts & Context switching

Interrupts can break isolation in various ways

Verified Interrupts & Context switching

Interrupts can break isolation in various ways

ARMv6m: Thread mode not set to privileged execution in certain ISRs #4246

 Closed  #4259

 vrindisbacher opened on Nov 21, 2024 · edited by vrindisbacher Edits ▾ Contributor ...

I'm opening this issue following the discussion in [#4242](#).

I believe that the following ISRs need to set the special `CONTROL` register to `0` before exception return, similar to [the `systick\_handler` on line 193](#)

- [Generic ISR](#)
 - I think that this is canceled out by [ARMv6m: Register clobbering in Generic ISR causes exception exit to return to process rather than Kernel #4245](#), since we return to the process rather than the kernel when coming from a process.
- [SVC Handler](#)
 - My hunch here is that we should be setting thread mode to privileged execution in the case we are coming from a process and need to return to the kernel to handle a syscall. I think setting the `CONTROL` register to `0` [here](#) should do the trick. Similar to how the `v7m svc handler` does [here](#).
 - It might also be a good idea to ensure that we set thread mode to unprivileged execution when we go from kernel to process. It seems like the `v7m svc handler` does this [here](#).



  alevy added **bug** on Nov 27, 2024

Verified Interrupts & Context switching

Lift ARM assembly to *Refined* Rust methods FLUXARM

Verified Interrupts & Context switching

Lift ARM assembly to *Refined* Rust methods FluxARM

ARM

```
systick_handler:
```

```
    mov r0, #0  
    msr CONTROL, r0  
    isb  
    ldr lr, =0xFFFFFFF9  
    bx lr
```



FluxARM

```
fn systick_handler(self: &mut Arm7[@old])  
    -> BV32[0xFFFF_FFF9]  
    ensures { ... }  
{  
    self.movw_imm(GPR::R0, 0);  
    self.msr(Control, GPR::R0);  
    self.isb();  
    self.ldr(lr(), 0xFFFF_FFF9);  
    self.bx(lr)  
}
```

Verified Interrupts & Context switching

Lift ARM assembly to *Refined* Rust methods FluxARM

ARM

```
systick_handler:
```

```
    mov r0, #0  
    msr CONTROL, r0  
    isb  
    ldr lr, =0xFFFFFFF9  
    bx lr
```



FluxARM

```
fn systick_handler(self: &mut Arm7[@old])  
    -> BV32[0xFFFF_FFF9]  
    ensures { ... }  
{  
    self.movw_imm(GPR::R0, 0);  
    self.msr(Control, GPR::R0);  
    self.isb();  
    self.ldr(lr(), 0xFFFF_FFF9);  
    self.bx(lr)  
}
```

Verify handlers save & restore kernel/MPU configuration

Verified *Isolation* in Tock [SOSP'25]

Verified *MPU configuration*

Verified *interrupts & context switching*

Verified *Isolation* in Tock [SOSP'25]

Component	Source	Fns(Trusted)	Specs(Trusted)
Kernel	11,974	550 (19)	355 (21)
ARM MPU	1,174	139 (14)	500 (17)
FLUX-STD	812	130 (48)	157 (60)
FLUXARM	3,413	95 (5)	1,908 (7)
Total	17,373	914 (86)	2,920 (105)

Specs = 6.5% lines of SLOC

Verified *Isolation* in Tock [SOSP'25]

Defensive programming in MPU driver API #4104

✓ Closed

🔗 #4135

ARMv6m: Register clobbering in Generic ISR causes exception exit to return to process rather than Kernel #4245

✓ Closed

🔗 #4259

ARMv6m: Thread mode not set to privileged execution in certain ISRs #4246

✓ Closed

🔗 #4259

Cortex-M MPU: `allocate_app_memory_region` allows access to kernel grant memory #4366

✓ Closed

🔗 #4384

App crashing allows potential sharing of kernel memory #4371

🔗 Open

Bug

Verification helped find & fix *six vulnerabilities* ...

Verified *Isolation* in Tock [SOSP'25]

Defensive programming in MPU driver API #4104

✓ Closed

🔗 #4135

ARMv6m: Register clobbering in Generic ISR causes exception exit to return to process rather than Kernel #4245

✓ Closed

🔗 #4259

ARMv6m: Thread mode not set to privileged execution in certain ISRs #4246

✓ Closed

🔗 #4259

Cortex-M MPU: `allocate_app_memory_region` allows access to kernel grant memory #4366

✓ Closed

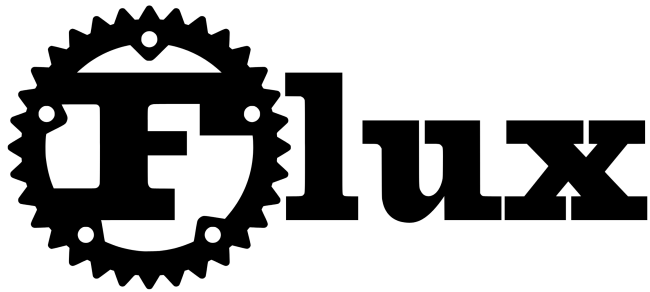
🔗 #4384

App crashing allows potential sharing of kernel memory #4371

🔗 Open

Bug

... and a *clearer* design with a *faster* kernel



1. Refinement

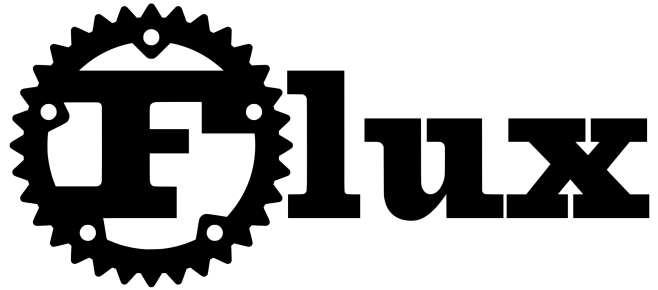
Index, Exist & Update

2. Types

Struct & Enum

3. Verified Systems

Isolation in Tock OS Kernel



1. Refinement

Index, Exist & Update

2. Types

Struct & Enum

3. Verified Systems

Isolation in Tock OS Kernel

flux-rs.github.io/flux/

110% ☆

Sign in

Show sidebar (^Z)

Introduction

Install & Run

About

Interactive Tutorial

1. Refinements

2. Ownership

3. Structs

4. Enums

5. Opaques

6. Consts

7. Externs

8. Traits

9. Iteration

10. Case Study: Simple Access Control

11. Case Study: Dynamic Access Control

12. Case Study: Dependent Typestates

13. Case Study: Neural Networks

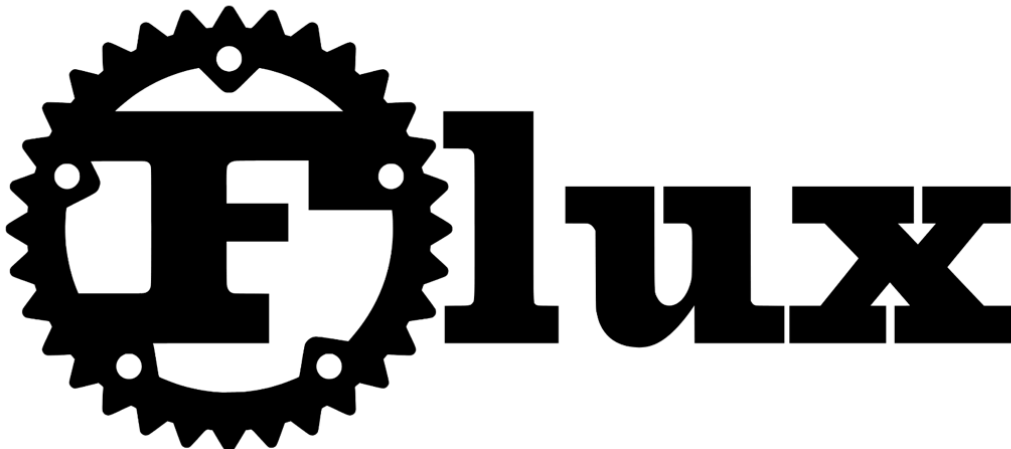
Appendix

14. Specifications

15. Developer's Guide

16. Architecture

The Flux Book



Flux is a [refinement type checker](#) plugin for Rust that lets you *specify* a range of correctness properties and have them be *verified* at compile time.

Flux works by extending Rust's types with [refinements](#): logical assertions describing additional correctness requirements that are checked during compilation, thereby eliminating various classes of run-time problems.

You can try it on the [online playground](#).

Better still, read the [interactive tutorial](#), to learn how you can use Flux on your Rust code.

1. Refinement

Index, Exist & Update

2. Types

Struct & Enum

3. Verified Systems

Isolation in Tock OS Kernel



<https://flux-rs.github.io>